System Verification and Validation Plan for EyeCanDo

Team 2, Hindsight
Mohammad Bilal
Ethan McMehen
Jackson Beach
Jake Finlay
Ahsan Muzammil

November 11, 2026

Table 1: Revision History

Date	Developer(s)	Change
2026/10/30	Mohammad Bilal	Completed section 3.3, individual reflection, & contributed to sections 1, 4.2, 6.1, 6.2, 7
2026/10/30	Ethan McMehen	Completed Section 2.3, Section 3.2, Section 3.7, Contributed to Section 1, Created Appendix 6.5, 6.6, 6.7
2026/10/31	Ahsan Muzammil	Completed section 2, 3.1, individual reflection, & contributed to sections 1, 4.1, 4.2, 6.4, 6.5, 6.6, 7
2026/10/31	Jake Finlay	Completed section 3.6
2026/10/31	Jake Finlay	Completed the Rewards and Scoreboard tests in section 4.1 and their test data in the appendix
2026/11/01	Jake Finlay	Completed the Look and Feel and the Usability and Humanity tests in section 4.2
2026/11/01	Jake Finlay	Completed individual reflection
2026/11/01	Jackson Beach	Completed Sections 3.4 and 4.1.4–4.1.6
2026/11/02	Ethan McMehen	Contributed to Section 4.1, 4.2, Edited Appendix 6.6, Added Reflection
2026/11/02	Jake Finlay	Completed the usability survey questions in the appendix
2026/11/02	Jackson Beach	Completed Sections 3.5 and 4.3, contributed to the AI Disclosure, and completed individual reflection

Contents

1 Symbols, Abbreviations, and Acronyms **v**

2 General Information **1**
2.1 Summary . . . 1
2.2 Objectives . . . 1
2.3 Custom Documents . . . 2
2.4 Relevant Documentation . . . 2

3 Plan **4**
3.1 Verification and Validation Team . . . 4
3.2 SRS Verification . . . 4
3.3 Design Verification . . . 5
3.4 Verification and Validation Plan Verification . . . 6
3.5 Implementation Verification . . . 6
3.6 Automated Testing and Verification Tools . . . 7
3.7 Software Validation . . . 7

4 System Tests **9**
4.1 Tests for Functional Requirements . . . 9
4.1.1 Setup . . . 9
4.1.2 Journey Map . . . 10
4.1.3 Level Content . . . 11
4.1.4 Instructions and Feedback . . . 14
4.1.5 Exiting and Interruptions . . . 17
4.1.6 Saving and Restoring Progress . . . 20
4.1.7 Rewards and Scoreboard . . . 22
4.1.8 Characters and Accessories . . . 24
4.1.9 Caregiver Features . . . 25
4.2 Tests for Nonfunctional Requirements . . . 26
4.2.1 Look and Feel Requirements . . . 26
4.2.2 Usability and Humanity Requirements . . . 27
4.2.3 Performance Requirements . . . 29
4.2.4 Operational and Environmental Requirements . . . 31
4.2.5 Maintainability and Support Requirements . . . 32
4.2.6 Security Requirements . . . 33
4.2.7 Cultural and Political Requirements . . . 34
4.2.8 Compliance Requirements . . . 35
4.3 Traceability Between Test Cases and Requirements . . . 37

5 Unit Test Description **40**
5.1 Unit Testing Scope . . . 40
5.2 Tests for Functional Requirements . . . 40
5.3 Tests for Nonfunctional Requirements . . . 40
5.4 Traceability Between Test Cases and Modules . . . 40

6 Appendix **41**
6.1 Symbolic Parameters . . . 41
6.2 Reward and Scoreboard Test Data . . . 42
6.3 Design Verification Checklist . . . 43
6.4 Usability Survey Questions . . . 43
6.5 SRS Checklist Questions . . . 45
6.6 Research Proxy to Stakeholder Requirement Validation . . . 46
6.7 Software Validation Checklist for Supervisor Validation . . . 47

7 Appendix — Generative AI Use Disclosure **49**
7.1 AI Tools Used . . . 49

7.2 How and Where Used 49
7.3 Verification of AI-Generated Content 50

8 Appendix — Reflection **52**

List of Tables

1 Revision History i
2 Symbols, Abbreviations, and Acronyms v
3 VnV Team Roles and Testing Responsibilities 4
4 Automated Testing and Verification Tools 7
5 Traceability Between Test Cases and Requirements 37
6 SRS Constants 41
7 Additional System-Test Parameters 42
8 Test Levels and Their Rewards 42
9 Saved States for the Rewards and Scoreboard Tests 43
10 Usability Survey Rating Statements 44
11 SRS Checklist Items and Their Verification Goal for Justification 45
12 Research Evidence Mapping to Stakeholder Requirement for Validation 47
13 Supervisor Validation Questions for the Rev 0 Demo - Version 1 48

1 Symbols, Abbreviations, and Acronyms

Table 2: Symbols, Abbreviations, and Acronyms

Symbol	Description
AI	Artificial Intelligence.
AV	Audiovisual; synchronized narration and animation.
CD	Custom Document; an additional course deliverable.
CPU	Central Processing Unit.
FPS	Frames per second.
FR	Functional Requirement.
ID	Identifier; used to identify screens, assets, or content.
LC	Likely Change, as defined in the SRS.
NFR	Non-Functional Requirement.
STR	Stakeholder Requirement.
UC	Use Case.
OS	Operating System.
REB	Research Ethics Board.
SBOM	Software Bill of Materials.
SRS	Software Requirements Specification.
ST-FR	System test for a functional requirement; numbered in order of appearance.
ST-NFR	System test for a non-functional requirement; numbered in order of appearance.
TA	Teaching Assistant.
VnV	Verification and Validation.
WCAG	Web Content Accessibility Guidelines.

For the remaining requirements terminology, see the definitions and abbreviations table in the SRS.

This document presents the Verification and Validation (VnV) plan for EyeCanDo, a game that helps children aged 3 to 7 prepare for eye appointments by introducing them to common examination procedures. It outlines how the team will verify that the software meets the requirements in the SRS and validate that it is suitable for its intended users. The document covers the planned reviews, system and unit tests, and validation activities. The appendices provide supporting checklists, questionnaires, and test parameters.

2 General Information

2.1 Summary

EyeCanDo is a mobile application designed to prepare children aged 3–7 for paediatric ophthalmology appointments. The application aims to reduce children's anxiety and improve their preparedness by introducing common eye examination procedures through age-appropriate interactive games.

The application follows a journey-based structure consisting of three preparation levels: Vision Testing, Eye-Drop Preparation, and Slit-Lamp Preparation. Each level focuses on a specific ophthalmology procedure and contains smaller interactive activities or exercises that familiarize children with what to expect during their appointments. Children can access and replay any preparation level at any time, regardless of their progress in other levels.

EyeCanDo provides visual demonstrations, narration, and encouraging feedback throughout gameplay. Children can select and customize a character, earn journey points and in-game rewards upon first completing each preparation level, and track their progress through a personal scoreboard. Completion status and rewards are tracked at the preparation level rather than for individual exercises.

Caregivers can access procedure guides, manage application preferences, and reset saved progress. The application operates offline after installation, stores progress locally, and does not require user accounts. EyeCanDo is intended solely as an educational preparation tool and does not provide medical advice, diagnosis, or treatment.

2.2 Objectives

The primary objective of this Verification and Validation (VnV) Plan is to build confidence that EyeCanDo meets its specified requirements and provides a safe, reliable, and age-appropriate preparation experience for children aged 3–7. Given the project's scope and available resources, verification and validation efforts will prioritize the following qualities:

- **Accuracy and Safety of Educational Content:** Verify that the preparation activities accurately represent common ophthalmology procedures, provide age-appropriate explanations, and avoid misleading or distressing information. Clinical content and applicable requirements will be reviewed with the project supervisor where feasible.

- **Functional Correctness and Reliability:** Verify that the application's core functionality behaves according to the SRS [1]. Particular attention will be given to ensuring that all preparation levels remain accessible, gameplay interactions function correctly, progress and rewards are recorded accurately, and the application operates reliably offline.

- **Preliminary Usability and Accessibility:** Assess whether the application's navigation, instructions, and interactions are understandable and appropriate for the intended age group. This will involve reviews by the development team and peers, including task-based inspections in which reviewers evaluate the application from the perspectives of the child and caregiver personas defined in the SRS [1].

The following validation objectives are outside the scope of the capstone project:

- **Direct Usability Testing with Primary Users:** Usability testing involving children and their caregivers will not be conducted during the capstone project, as the required research ethics approval will not be pursued within the project timeline. Instead, preliminary usability assessments will be performed through peer reviews and persona based inspections. Since the reviewers are not the intended users, their feedback may not accurately reflect how children and caregivers would interact with the application. Therefore, these assessments can help

identify potential usability issues but cannot fully confirm the application's usability for its intended users.

- **Clinical Effectiveness Validation:** Determining whether EyeCanDo reduces children's anxiety, improves cooperation during examinations, or increases clinical efficiency requires studies involving patients and healthcare professionals. Such studies are outside the scope of this project and may be pursued after the capstone, subject to research ethics approval.

- **Exhaustive Compatibility and Third-Party Verification:** Testing will focus on the devices and operating systems available to the team. Exhaustive testing across all supported device configurations and independent verification of third party software will not be performed due to resource constraints. Third party components will instead be assessed through their integration with the application.

The team will prioritize verification through software testing, code and document reviews, and feedback from the supervisor where appropriate. Reviews conducted by classmates using the child and caregiver personas will help identify potential usability issues. However, these reviews cannot fully confirm how actual children and caregivers would interact with the application. The team will document these limitations, while usability testing with the intended users and evaluating the application's effectiveness in reducing anxiety will be left for future work.

2.3 Custom Documents

The team plans to complete two Custom Documents (CDs) as part of the EyeCanDo project, one during the Fall semester and one during the Winter semester. For the Fall semester, the team is will write Cognitive Walkthrough Report replacing the previously selected Client Interview Report. A Cognitive Walkthrough Report would evaluate whether users can understand and navigate the application's interface by having reviewers complete representative tasks from the perspectives of the child and caregiver personas. This change reflects the project's revised scope, as direct testing and interviews involving children and caregivers will not be conducted during the capstone project due to research ethics requirements.

For the Winter semester, the team currently plans to complete a Usability Report. This report would document preliminary usability evaluations conducted by team members and peers, focusing on the application's navigation, instructions, and ease of interaction. Since these evaluations will not involve the intended users directly, the results will be used to identify potential usability issues rather than to confirm usability for children aged 3–7. The selection of the Winter Custom Document is also subject to change following further discussion with the course professor and project supervisor. Any approved changes will be reflected in the project documentation and VnV Plan.

2.4 Relevant Documentation

The following project documents provide the requirements, design information, and development context used to plan and conduct verification and validation activities for EyeCanDo.

- **Software Requirements Specification (SRS):** The SRS [1] defines the functional and non-functional requirements of EyeCanDo, including preparation activities, journey progression, accessibility, and reliability. It serves as the primary reference for developing test cases and determining whether the application meets its specified requirements.

- **Problem Statement and Goals:** The Problem Statement [2] identifies the problem EyeCanDo aims to address, its intended users, and the project's overall goals. It provides the context for evaluating whether the application supports its intended purpose, while recognizing that measuring actual anxiety reduction and clinical effectiveness is outside the scope of this capstone project.

- **Development Plan:** The Development Plan [3] describes the team's development workflow, responsibilities, tools, and project schedule. It helps ensure that the planned verification and validation activities are feasible within the available time and resources.

- **Hazard Analysis:** The Hazard Analysis [4] identifies potential risks associated with the application and defines corresponding safety and security requirements. These findings will guide verification activities related to safe interactions, accurate educational content, data handling, and reliable application behaviour.

- **Design Document:** The Design Document will likely consist of a Design Specification Document describing the architecture, main components, interfaces, and expected behaviour of EyeCanDo. It will serve as a reference for verifying that the proposed design supports the functional and non-functional requirements defined in the SRS. The document will also be used during design verification to assess the consistency, organization, and correctness of the system design.

The selected Fall and Winter Custom Documents will also contribute to the VnV process by documenting preliminary interface and usability evaluations. Their findings will help identify design and usability issues that can be addressed during development. References to these documents will be added once their selection and content have been finalized.

3 Plan

This section outlines how the Hindsight team will carry out verification and validation throughout the capstone project. It identifies the team members responsible for these activities and explains how the SRS, design documents, VnV plan, and implementation will be reviewed. It also describes the testing tools and how the game will be validated with its intended users.

3.1 Verification and Validation Team

Verification and validation activities for EyeCanDo will be shared among the five members of Team 2, Hindsight. Each member has been assigned functional and nonfunctional requirement categories for testing based on the SRS [1]. These assignments are informed by the team's development roles [3], but verification will remain a collaborative effort.

Each member will coordinate the development and execution of test cases for their assigned requirement categories, document testing results, and report identified defects. However, these assignments do not imply exclusive responsibility for verifying a particular part of the application. Team members will also review one another's test cases, implementation, and results where feasible.

Responsibilities may vary between SRS verification, design verification, and implementation verification to encourage independent assessment. The project supervisor will provide feedback on selected requirements and educational content where appropriate. Classmates may also participate in preliminary usability evaluations using the child and caregiver personas defined in the SRS.

Table 3 summarizes each member's development role, assigned testing categories, and associated verification responsibilities.

Table 3: VnV Team Roles and Testing Responsibilities

Role	Member Name	Testing Areas (FR/NFR)	VnV Responsibilities
Technical Manager	Ahsan Muzammil	**FR:** Setup (4.1.1), Journey Map (4.1.2), Level Content (4.1.3). **NFR:** Cultural and Political (4.2.7).	Test setup, level accessibility, and procedure content. Coordinate testing and defect tracking.
Lead Front end Developer	Jake Finlay	**FR:** Rewards and Scoreboard (4.1.7). **NFR:** Look and Feel (4.2.1), Usability and Humanity (4.2.2).	Test interface presentation, rewards, and usability requirements.
Lead Back end Developer	Mohammad Bilal	**NFR:** Performance (4.2.3), Operational and Environmental (4.2.4), Maintainability and Support (4.2.5), Security (4.2.6).	Test performance, reliability, maintainability, and security.
Lead Researcher	Jackson Beach	**FR:** Instructions and Feedback (4.1.4), Exiting and Interruptions (4.1.5), Saving and Restoring Progress (4.1.6).	Test instructional behaviour, interruption handling, and progress recovery.
Team Liaison	Ethan McMehen	**FR:** Characters and Accessories (4.1.8), Caregiver Features (4.1.9). **NFR:** Compliance (4.2.8).	Test customization, caregiver features, and compliance. Coordinate external feedback.
Clinical Reviewer	Project Supervisor	**Clinical Review:** Selected requirements and educational content.	Provide feedback on clinical accuracy, age-appropriateness, and procedure representations where feasible.

3.2 SRS Verification

The SRS will be verified from three perspectives: the TA offering the course expectations perspective, classmates offering the readability and clarity from a novice perspective, and the project supervisor offering the clinical domain expertise perspective. All reviewers will have access to the SRS checklist (see Appendix 6.5), which covers completeness, consistency, correctness, testability, traceability to stakeholder needs, and clarity. Reviewers may use the full checklist or only the items relevant to their perspective.

TA and Peer Review Verification: The TA's formal feedback will be logged by the team as GitHub issues labeled TA Feedback. Classmates will submit their feedback directly as GitHub issues labeled Peer Review, focusing on readability, clarity, and whether the requirements can be understood without prior knowledge of the domain. The team will sort every issue, assign an owner, and close it with a documented resolution that can be a change to the SRS or a recorded reason for no change.

Supervisor Review Meeting: Given the length of the SRS (90+ pages), the supervisor review is designed so that reading the entire document is not required. The SRS and checklist will be shared with the supervisor one week before the meeting, along with a request to read stakeholders and scope, the use cases, and the functional requirements. The meeting will last approximately 45 minutes maximum and will be led by the team liaison, with a second team member taking notes. The meeting has three stages, each with a distinct purpose:

1. **Stage 1 Walkthrough:** The team will perform a walkthrough of the SRS document with the supervisor to briefly present the project scope, stakeholders, system context, and assumptions. The goal is to verify document clarity, receive verification feedback, and receive information on misunderstandings regarding the specifications intent in the context of the sections listed.

2. **Stage 2 SRS Checklist Review:** The team and supervisor will work through the SRS checklist together, confirming that the required elements are present, that requirements do not conflict, that terminology is used consistently, and that functional and NFRs trace back to stakeholder needs and project goals. The goal is to verify completeness and traceability.

3. **Stage 3 Task-based Inspection:** The supervisor will be asked to complete two tasks. The first is to select two use cases and confirm that the steps and exceptions match real clinical workflow. The second is to select two NFRs and describe how to measure the criterion to verify the measurable criteria is aligned with clinical measuring expectations. The goal is to verify consistency, testability and use case correctness within the clinical domain.

All feedback from the meeting will be recorded in a single GitHub issue under the SM: Stakeholder + Supervisor Meetings GitHub milestone. The team will then divide it further into individual issues, so that each can be assigned and tracked. All verification issues will be resolved and closed before the Rev 0 presentations begins on February 1, 2027 and the supervisor will be informed of how her feedback was addressed.

3.3 Design Verification

Design verification will be carried out after the design document is completed. The purpose is to check that the architecture described in the design document is consistent with the requirements in the SRS and address the relevant concerns identified in the Hazard Analysis. The design will be verified through three reviews:

1. **Structured Internal Review:** All members of the Hindsight team will review the design document, preferably in a group meeting. One team member will be appointed to lead the review and work through the checklist in Appendix 6.3. The team will discuss any concerns, record them as GitHub issues, and assign members to address them.

2. **External Peer Review:** Members of at least one other capstone team will review the design as part of the course's milestone peer review process. The checklist in Appendix 6.3 will be shared with the reviewers, but they will not be required to use it. Their findings will be submitted as GitHub issues for the team to review and address.

3. **Teaching Assistant Feedback:** The teaching assistant (TA) will evaluate the design documents as part of the course deliverable assessment. Any feedback requiring changes will be recorded as GitHub issues and addressed by the team.

For each review, GitHub issues will identify the relevant section of the design, the problem found, and the change needed. The assigned team member will make the correction, and another team member will check the update before the issue is closed.

3.4 Verification and Validation Plan Verification

The Verification and Validation Plan will be reviewed by the team before submission and after major revisions to check completeness, accuracy, and feasibility. Each section will be checked by a team member besides its author. Colleagues from at least one other team will also review the plan through the peer review process. This checklist will guide the review:

☐ Are all sections complete, with any deferred or excluded work explained?

☐ Does every functional and non-functional requirement have a planned test or review, and does the traceability table match the tests?

☐ Does each test specify its initial conditions, inputs, steps, expected results, and evidence?

☐ Are expected results and test parameters consistent with the SRS and Hazard Analysis (and with the design document when available)?

☐ Are testing, reviews, and other validation activities feasible with the available time, equipment, and participants?

☐ Are terminology, test identifiers, and references consistent and correct?

Feedback will be tracked through GitHub issues. Each issue will identify the affected section, the problem, and the proposed change, and will be assigned to a team member. Corrections will be committed with reference to the issue and checked by another member before it is closed. If a suggestion is not adopted, the team will record the reason why in the issue.

3.5 Implementation Verification

Implementation verification will check that the code follows the design and meets the requirements in the SRS [1]. The team will use dynamic testing to check the game's behaviour and static review to identify problems without running the game.

The system tests in Sections 4.1 and 4.2 define the procedures and expected results for checking gameplay, saved progress, rewards, performance, and other required behaviour. Manual tests will use the available iOS and Android devices, while automated Unity Play Mode tests will check interactions such as awarding rewards and updating the scoreboard. Static inspections listed in Section 4.2 will also be carried out, including reviews of game content and dependencies.

Following the Development Plan [3], every code change will be reviewed by at least one other team member through a GitHub pull request. Reviewers will check correctness, error handling, and adherence to the team's C# and Unity coding standards. For critical logic, such as saving progress and granting rewards, the author and reviewer will walk through the code together and compare it with the SRS and design. Reviewers will also check aspects that tools cannot judge, such as whether comments explain important decisions and whether game logic, UI logic, and data management are separated appropriately.

GitHub Actions will run formatting checks, static analyzers, automated tests, and build verification on pull requests using the tools in Section 3.6. Required checks must pass before a merge.

Test results will record the test identifier, build version, device and operating system where applicable, and supporting evidence. Defects found through testing or review will be tracked as GitHub issues and assigned to a team member. After a fix, the failed test and related tests will be repeated to check for regressions. Dependency changes will require the full functional test suite on a release build.

3.6 Automated Testing and Verification Tools

EyeCanDo is written in C# with the Unity engine. The Development Plan [3] already sets out the main tool choices. Section 8.3 describes the GitHub Actions pipeline, Section 11 lists the unit testing framework, and Section 13 gives the coding standard and how it is enforced. This section summarizes the tools that automate testing, formatting, and linting, and adds the plan for code coverage. Table 4 lists each tool. These choices may change as implementation progresses. Any change will be recorded in the revision history of this document and the Development Plan.

Table 4: Automated Testing and Verification Tools

Purpose	Tool	Use
Unit testing	Unity Test Framework (NUnit), Edit Mode tests	Tests game logic that does not need a running scene, such as level progression, quiz scoring, and saved progress.
Unity interaction testing	Unity Test Framework, Play Mode tests	Tests scene loading, touch input handling, and component behaviour across frames.
Continuous integration	GitHub Actions with GameCI (`unity-test-runner`, `unity-builder`)	Runs the format check, linting, both test suites, and a build on every pull request. All checks must pass before a merge (Development Plan, Section 8.3).
Formatting	`dotnet format` with a shared `.editorconfig`	Fails CI when code breaks the formatting or naming rules in Development Plan Section 13.
Linting	.NET Roslyn analyzers and Microsoft.Unity.Analyzers	Reports common C# and Unity mistakes, such as unused code and inefficient Unity API calls. Rider inspections give the same feedback locally.
Code coverage	Unity Code Coverage package	Records line and method coverage during CI test runs.

Formatting and Linting

The naming rules in the Development Plan, such as PascalCase methods and camelCase private fields, will be written as `.editorconfig` rules. CI will run `dotnet format` in verification mode against the project files that Unity generates, so any violation fails the pull request. The analyzer rules will be configured in the same `.editorconfig`. Rules that catch real defects will be set to errors and fail the build. Style suggestions will stay as warnings. Reviewers will still check the parts of the standard that tools cannot, such as comment quality and separation of game logic from UI logic.

Code Coverage

GameCI will run the Edit Mode and Play Mode tests with the Unity Code Coverage package enabled. Coverage will be measured only for the team's game assemblies. Test code, Unity packages, and Asset Store content will be excluded. Each CI run will produce an HTML report and a summary of line and method coverage for each assembly. The summary will be shown on the pull request and the full report saved as a workflow artifact. For each milestone, the team will record the coverage of each assembly in the VnV Report and list any modules with little or no coverage. The target is `COVERAGE_LINE_TARGET` line coverage for game-logic assemblies. UI and scene code will rely more on Play Mode and manual system tests. Coverage will be used to find untested code, not as proof of correctness.

Performance is not measured in the pipeline. The Unity Profiler is used manually during the performance system tests, such as ST-NFR-13 and ST-NFR-18.

3.7 Software Validation

This plan complements the SRS verification plan in Section 3.2, which checks that the requirements are correct, whereas validation checks that the right product is built. However, validation with children and parents is not planned because research ethics approval (HiREB) for work with children does not fit the capstone timeline, as justified in Section 2.2 deferring validation from direct usability testing until HiREB ethics approval. Instead, EyeCanDo will be validated through three proxies:

- Published studies of play-based preparation and anxiety reduction in children will be used to check that the design is evidence-based, with each study mapped to the stakeholder requirement it supports. Version 0 of Table 12 is included in Appendix 6.6 to demonstrate the mapping planned format of a research source to the STR.

- Child-life specialists and child development experts, where available, will use their professional knowledge to judge how children aged 3 to 7 would perceive the game, following a scripted walkthrough guided by the project personas. Their feedback cannot confirm reduced anxiety or child usability requirements (STR2, STR3, STR18), and those remain unvalidated until the pilot.

- Shortly after the Rev 0 demo, the team will demonstrate the product to the supervisor using a scripted scenario and targeted questions to validate it against clinical needs outlined by the stakeholder requirements and use cases in the SRS. Version 1 of the targeted questions have been developed and included in Appendix 6.7 but Table 13 is subject to change to include more questions up until Rev 0.

The team will also complete two checks that need no participants. First, a persona-based walkthrough, in which the team uses EyeCanDo as each project persona and records anything that conflicts with their goals or frustrations. Second, an accessibility audit against WCAG 2.2 Level AA to support STR17. This audit will use the platform accessibility tools (iOS Accessibility Inspector and Android Accessibility Scanner), manual checks, and simulations of colour vision deficiency, blurred vision, and muted audio. However, these methods cannot replace testing with children who have visual or hearing impairments, so full validation of STR17 also remains deferred until a future HiREB approval. All feedback will be logged as GitHub issues resolved before Rev 1.

4 System Tests

This section describes the system tests used to verify the functional and non-functional requirements in the SRS. The functional tests check that the main features of the game work as expected, while the non-functional tests check qualities such as performance, offline use, maintainability, and security. Each test explains what will be checked and how the result will be recorded.

4.1 Tests for Functional Requirements

The tests in this section verify the functional requirements (FRs) defined in the SRS. These requirements describe the features and behaviours the game must provide. The tests are organized according to the FR categories in the SRS, covering setup, journey map, level content, instructions and feedback, exiting and interruptions, saving and restoring progress, rewards and scoreboard, characters and accessories, and caregiver features. Each test identifies the requirements it covers, the initial conditions, expected results, and how the test will be performed.

For the Capstone course, functional testing will primarily be performed manually using the available test devices. Testers will record the device, operating system, build version, and test results. Any limitations or changes to the planned testing procedures will be documented under the relevant test.

4.1.1 Setup

These tests cover the first-use medical notice and the locally saved caregiver acknowledgement in FR1–FR2 of the SRS [1]. For tests requiring a first launch, use a test installation with its application data cleared and no restored backup. Record the device, operating-system version, build identifier, and observed result.

ST-FR-1: Medical Notice Before First Use

Type: Functional, Dynamic, manual.

Requirements Covered: FR1.

Initial State: The current build is installed on a test device. All application data has been cleared, and no caregiver acknowledgement is stored.

Input/Condition: Launch the app. Read the medical notice and attempt to enter the home screen without selecting its acknowledgement control. Close and reopen the app without acknowledging the notice.

Output/Result: The notice identifies the game as an educational preparation aid and states that it does not replace professional medical advice, diagnosis, or treatment. The home screen remains inaccessible until acknowledgement. The notice is shown again after relaunch.

Test Case Derivation: FR1 requires the notice before first use and prevents access to the home screen until the caregiver acknowledges it.

How test will be performed: On a clean installation, capture the notice and check all required statements. Attempt any available back, home, or close-navigation action without acknowledging. Relaunch and confirm the notice is still required. Record the available controls and whether any route bypasses acknowledgement.

ST-FR-2: Medical Notice in Caregiver Guide

Type: Functional, Dynamic, manual.

Requirements Covered: FR1.

Initial State: Initial setup has been completed and the home screen is accessible.

Input/Condition: Open the caregiver procedure guide and select the available medical notice or educational-use disclaimer.

Output/Result: The medical notice remains accessible and states that the game is for educational preparation, not medical advice, diagnosis, or treatment. Viewing it does not change journey progress.

Test Case Derivation: FR1 explicitly requires the same notice to remain available from the caregiver procedure guide.

How test will be performed: Navigate to the caregiver guide, open the notice, compare its required statements with those checked in *ST_AM_NoticeFirstLaunch*, and return home. Record the navigation path and capture the displayed notice.

ST-FR-3: Acknowledgement Persists Across Launches

Type: Functional, Dynamic, manual.

Requirements Covered: FR2.

Initial State: A clean installation is displaying the first-use medical notice; no acknowledgement has been recorded.

Input/Condition: Select the acknowledgement control once, complete any introductory screens, reach the home screen, fully close the app, and launch it again. Repeat the close-and-launch operation once.

Output/Result: The acknowledgement is saved locally. Both subsequent launches reach the home screen without requesting acknowledgement again. No account or network connection is required to retain it.

Test Case Derivation: FR2 requires a locally stored acknowledgement and omission of the notice on later launches.

How test will be performed: Record the acknowledgement action, then force-close and relaunch the app twice. Confirm that the home screen is reached without a repeated notice. If a development build exposes the acknowledgement field, also check that it changes from false to true.

4.1.2 Journey Map

These tests verify FR3–FR5: presentation of the three procedure levels, independent access to every level, and replay without loss of a completed status. The names, order, and allowed statuses come from the SRS [1]; no custom image or data file is needed because the test uses the levels and illustrations packaged in the release build.

ST-FR-4: Journey Map on a New Profile

Type: Functional, Dynamic, manual.

Requirements Covered: FR3.

Initial State: Initial setup is acknowledged, and a new journey has zero completed or started levels. If necessary, use the caregiver's journey-reset control before starting the test.

Input/Condition: From the home screen, open the journey map. Do not start any level.

Output/Result: Exactly the three defined procedure levels are shown in level-number order: Vision Testing, Eye-Drop Preparation, and Slit-Lamp Preparation. Each has a name, an illustration, and a visible *Not Started* status. No level is displayed as locked.

Test Case Derivation: FR3 specifies one illustrated, named, status-labelled map entry per procedure, ordered by level number.

How test will be performed: Reset journey progress, open the map, and record the order, titles, images, status labels, and availability of all three entries in a screenshot and checklist.

ST-FR-5: Map Displays Three Different Statuses

Type: Functional, Dynamic, manual.

Requirements Covered: FR3, FR4, FR12.

Initial State: Initial setup is complete; journey progress is reset so all three levels are *Not Started*.

Input/Condition: Start Vision Testing, then use its home control before the ending. Next start Eye-Drop Preparation and reach its end-of-level screen. Do not open Slit-Lamp Preparation. Return to the journey map.

Output/Result: Vision Testing is *In Progress*, Eye-Drop Preparation is *Completed*, and Slit-Lamp Preparation is *Not Started*. All three entries remain selectable.

Test Case Derivation: FR3 requires the map to show the current status of each level; FR4 and FR11 define the changes on start and finish.

How test will be performed: Follow the stated sequence on a reset journey, recording each transition.

Compare all three status indicators with the expected state and capture the final journey map.

ST-FR-6: Start Any Level Without Unlocking

Type: Functional, Dynamic, manual.

Requirements Covered: FR4.

Initial State: Journey progress is reset; all levels are *Not Started*, and journey points and coins are zero.

Input/Condition: Select Slit-Lamp Preparation first and activate its play control. Before the ending, return to the map. Then select Eye-Drop Preparation, activate play, and return early. Finally select Vision Testing and activate play. For each start, record the first displayed level step.

Output/Result: Each selected level starts at its beginning without requiring a completed prerequisite, journey points, or coins. Its status becomes *In Progress*; the other levels retain their existing statuses. No reward is given merely for starting a level.

Test Case Derivation: FR4 allows every level to start regardless of progress or rewards, applies the start operation, and requires the level to begin from its first step.

How test will be performed: Execute the three starts in the specified order. After each early exit, inspect the journey map and record the starting scene, all three statuses, and displayed reward balances. Flag any lock, prerequisite, or resumed mid-level scene.

ST-FR-7: Replay Does Not Remove Completion

Type: Functional, Dynamic, manual.

Requirements Covered: FR5, FR4.

Initial State: On a reset journey, Vision Testing has been played to its end once and shows *Completed*. Record the current journey points, coins, and level status.

Input/Condition: Select the completed Vision Testing level, start its replay, exit before the ending, and inspect the map. Start Vision Testing again and play to the end.

Output/Result: Both replays start from the beginning. Vision Testing remains *Completed* during and after replay, including after early exit. Its original completion reward is not awarded again; recorded points and coins remain unchanged by replay.

Test Case Derivation: FR5 preserves completed status during replay, while FR4 restarts the level from the beginning. The SRS formal progress model permits first-completion rewards only once.

How test will be performed: Capture the map and reward balances before replay, after early exit, and after replay completion. Compare these values and record whether each replay began at the introduction.

4.1.3 Level Content

These tests verify FR6–FR12 using the three levels and their narration, images, and animations. Before testing, the tester will record the build version and prepare a content inventory of relevant assets, scripts, age-group variants, and procedure steps, while clinical accuracy will be reviewed separately in the NFR tests.

ST-FR-8: Procedure Content in Every Level

Type: Functional, Dynamic, manual content inspection.

Requirements Covered: FR6.

Initial State: Initial setup is complete and the three procedure levels are available with the final content inventory.

Input/Condition: Open Vision Testing, Eye-Drop Preparation, and Slit-Lamp Preparation separately. For each, advance through all built-in screens and animations, including any optional exercises.

Output/Result: Each level presents at least one identifiable step, piece of equipment, or instruction for its own procedure: Vision Testing includes an eye-chart or symbol-identification activity; Eye-Drop Preparation depicts or explains clinician-administered drops; and Slit-Lamp Preparation depicts or explains the slit-lamp device or patient positioning. None is an unrelated activity without procedure

information.

Test Case Derivation: FR6 requires each level to prepare for its corresponding procedure by presenting at least one relevant part. The three use cases describe the intended procedure examples.

How test will be performed: Play each level from its beginning, record the first qualifying procedure moment and its asset or screen ID, and capture a screenshot or narrated-screen recording. Compare the three observations with the approved procedure inventory.

ST-FR-9: Age-Specific Content Selection

Type: Functional, Dynamic, manual.

Requirements Covered: FR7.

Initial State: The caregiver can access preferences. Before testing, the content inventory states which levels, if any, have a 3–4-year variant, a 5–7-year variant, and default content.

Input/Condition: Set the age group to 3–4 years and start each level from the beginning; record its opening content ID. Repeat with age group 5–7 years. Finally use the unset age-group option, or clear local preferences and complete setup again if no unset control is exposed, then start each level again.

Output/Result: For each run, a level presents its matching age-specific content when that variant exists. Where no matching variant exists, or the age group is unset, it presents the listed default content. No level becomes unavailable because of the setting.

Test Case Derivation: FR7 defines the values *3–4*, *5–7*, and *unset*, and requires a matching variant when provided, with fallback to default otherwise.

How test will be performed: Use the caregiver preferences to run the stated three-setting matrix across all three levels. Compare each displayed content ID with the recorded content inventory; record all nine setting-and-level combinations. Do not mark a missing optional age-specific variant as a failure if its default content is used.

ST-FR-10: Required Sensation Explanation Cannot Be Skipped

Type: Functional, Dynamic, manual.

Requirements Covered: FR8, FR9.

Initial State: The current build includes the approved procedure narration and animations. Identify every scene that depicts a potentially felt sensation (for example, stinging from eye drops or bright light during a slit-lamp examination) in the content inventory.

Input/Condition: For each identified scene, start its level from the beginning and use the available skip control as soon as it appears. Repeat once without skipping the optional animation or exercise.

Output/Result: On both paths, the child receives the relevant sensation description before the sensation is depicted or the optional part is skipped. The mandatory description itself cannot be bypassed. No narration, text, or image promises that the procedure will be painless or free of discomfort.

Test Case Derivation: FR9 requires honest advance explanations of sensations and FR8 prohibits skipping the description, even when the corresponding exercise or animation is skipped.

How test will be performed: Record both paths for every scene listed in the inventory. Note the first frame or narration cue for the description, the skip action, and the first frame depicting the sensation. Review all text and narration for prohibited reassurance. Save the recordings and a pass/fail result for every applicable scene.

ST-FR-11: Differences Between Game and Clinic

Type: Functional, Dynamic, manual.

Requirements Covered: FR10.

Initial State: All three levels contain the current approved narration and pictures. Device audio is enabled.

Input/Condition: Play each level from beginning to end, including its introductory and concluding content.

Output/Result: Each of the three levels communicates, through both narration and pictures, that a

real doctor may perform the procedure somewhat differently from the game. The message is present in Vision Testing, Eye-Drop Preparation, and Slit-Lamp Preparation.

Test Case Derivation: FR10 explicitly requires this message in both narration and pictures in every level.

How test will be performed: For each level, identify the narration clip or on-screen spoken message and the corresponding picture or visual cue. Record both content IDs and a screenshot or recording. Mark a level as failing if either channel is missing.

ST-FR-12: Permission to Tell the Doctor About Discomfort

Type: Functional, Dynamic, manual.

Requirements Covered: FR11.

Initial State: All three levels are accessible and audio is enabled.

Input/Condition: Complete each level once normally. Reset the journey, then complete each level using skip for every optional exercise or animation that offers it.

Output/Result: Before the ending in both paths for all three levels, the child is told that they may tell the doctor if they feel worried or uncomfortable. Skipping exercises does not remove this message.

Test Case Derivation: FR11 requires the reassurance before the end of every level. The alternative skip path checks that a child cannot miss it by bypassing an optional exercise.

How test will be performed: Record the normal and skip-heavy runs for all three levels, identify the screen or narration ID conveying the message, and confirm that it occurs before each end-of-level screen.

ST-FR-13: Completion Recorded Only at Level Ending

Type: Functional, Dynamic, manual.

Requirements Covered: FR12.

Initial State: Journey progress is reset so all three levels are *Not Started*.

Input/Condition: Start Vision Testing and exit before its ending. Return to the map. Then play Vision Testing to its end, followed by Eye-Drop Preparation and Slit-Lamp Preparation to their respective endings. Record the map after each step.

Output/Result: Early exit from Vision Testing leaves it *In Progress* without an end-of-level screen or completion reward. Reaching the ending of each level displays its end-of-level screen and changes only that level's status to *Completed*. All three are *Completed* only after their own endings have been reached.

Test Case Derivation: FR12 applies the finish operation at level ending and changes the level status to *Completed*. The formal model does not count individual exercises as separate levels.

How test will be performed: Follow the specified exit and three-completion sequence. Capture the ending of each level and the map after each exit or completion; compare all three statuses after every transition.

ST-FR-14: Skipped Exercises Do Not Prevent Level Completion

Type: Functional, Dynamic, manual.

Requirements Covered: FR12, FR8, FR9, FR11.

Initial State: Journey progress is reset, and the three levels have their normal content and available skip controls.

Input/Condition: Start each level. For every optional exercise, round, or animation that offers skip, select skip instead of completing it; continue through any mandatory educational narration and reach the level ending.

Output/Result: Each level reaches its end of level screen and becomes *Completed*, despite skipped optional activities. Mandatory sensation descriptions and the message about telling a doctor about worry or discomfort are still presented. No individual skipped exercise is incorrectly treated as a separate unfinished level.

Test Case Derivation: The SRS tracks completion per procedure level, not per exercise; FR12 requires completion at the ending. FR8, FR9 and FR11 protect mandatory educational information.

How test will be performed: Record a skip heavy run through all three levels. Count and note each selected skip, verify every mandatory message, and capture each end of level screen and final journey map status.

4.1.4 Instructions and Feedback

These tests verify FR13-FR21. Extend the content inventory from Section 4.1.3 with child-facing instructions, exercises, questions and reviewed answers, animations, and screen states and transitions. Include instructions outside levels and check all levels' default content and each distinct age-group variant.

Use walkthroughs and the Unity test tools in the Development Plan, with temporary input, playback, and storage logs where needed. Record the screen or content ID with supporting recordings or logs.

ST-FR-15: Narrated Instructions with Demonstrations

Type: Functional, Dynamic, manual.

Requirements Covered: FR13.

Initial State: Setup is complete and narration is audible.

Input/Condition: Follow every listed child-facing instruction using only narration and demonstrations. Include hints, answer feedback, and instructions outside levels.

Output/Result: Every instruction has matching narration and a visual demonstration. No required action depends on reading.

Test Case Derivation: FR13 requires both forms of guidance and prohibits levels from requiring reading.

How test will be performed: Record the walkthrough with audio. Match each instruction ID to its narration, demonstration, and action; note missing or conflicting guidance and any reliance on written text.

ST-FR-16: Hints after Incorrect Actions

Type: Functional, Dynamic, manual.

Requirements Covered: FR14.

Initial State: An exercise awaits an action identified in the inventory. Wrong answers to questions are covered by ST-FR-19.

Input/Condition: Make an incorrect action, repeat it, then perform the requested action. Cover each distinct incorrect response path, such as placing or releasing an object outside its target. Unrelated touches are checked in ST-FR-30.

Output/Result: Each incorrect action gives an encouraging hint and allows another attempt. The correct action is accepted and play continues.

Test Case Derivation: FR14 requires hints and retries for incorrect actions, including repeated mistakes.

How test will be performed: Record each action, the hint's wording and visual guidance, and the successful retry. Check that the hint guides without blame or discouragement.

ST-FR-17: Visual Cues Remain Available

Type: Functional, Dynamic, manual.

Requirements Covered: FR15.

Initial State: Setup is complete. Visual-cue size and reduced-motion settings are available under FR44.

Input/Condition: Inspect every listed level state and transition, including after skips and restarts. Try changing or disabling cues through child-facing controls, and accessing cue settings before and after the parental gate. Check minimum and maximum cue sizes and reduced motion on and off, varying one setting at a time, then combining minimum size with reduced motion.

Output/Result: Cues remain visible and provide guidance in every state. The child cannot disable

them; settings that change them require the parental gate. Size and motion changes do not remove needed guidance.

Test Case Derivation: FR15 requires persistent cues and restricts changes to caregiver settings under FR41.

How test will be performed: Mark each state and setting combination in the inventory. Record missing cues and whether each route to cue settings requires the gate.

ST-FR-18: Instructions without Sound

Type: Functional, Dynamic, manual.

Requirements Covered: FR16.

Initial State: Setup is complete. Confirm audible narration before each sound condition and identify device mute modes that silence media, not just calls or alerts.

Input/Condition: Follow every listed instruction in a silent walkthrough, including hints, reveals, and sensation descriptions. Exercise each mute-detection and prompt path using device mute, zero media volume, and zero caregiver narration volume. Cover silence at entry and loss of sound during playback. Leave the prompt unanswered and finish the level visually, then restore sound using the indicated control and play the next instruction.

Output/Result: Pictures, icons, and demonstrations convey every instruction. The caregiver prompt explains how to restore sound without blocking play. Muting does not pause play or prevent completion; restoring sound permits subsequent narration while visual guidance remains available.

Test Case Derivation: FR16 requires visual guidance and a non-blocking prompt. SR-18 of the Hazard Analysis [4] also covers narration disabled through settings.

How test will be performed: Verify silence at the device output. Record the guidance, prompt, completion, and sound restoration. Record unavailable mute modes as unverified.

ST-FR-19: Correct Answers after a Wrong Answer

Type: Functional, Dynamic, manual with timing logs.

Requirements Covered: FR17, FR19; reveal cues and timing from NFR10 and NFR12.

Initial State: A question is displayed with audible narration and a reviewed answer key.

Input/Condition: Select a wrong answer. Wait until the reveal finishes and any configured timeout passes; with no timeout, wait `ANSWER_RETRY_OBSERVATION_S` seconds after the reveal. Then select a correct answer. Across attempts, cover every wrong and revealed correct option at least once, pairing selections to avoid redundant combinations. Also test a wrong answer followed by holding skip without a correct selection.

Output/Result: All correct answers, and no incorrect ones, are marked within `ANSWER_REVEAL_MAX_S` second, using narration and at least two non-colour cues. The reveal remains available for retry. Selecting a correct answer gives encouraging feedback before continuation; skip permits continuation without a correct answer or penalty.

Test Case Derivation: FR17 requires reveal and retry; FR19 permits skipping. NFR10 and NFR12 specify cues and timing, also required by SR-16 of the Hazard Analysis [4].

How test will be performed: Compare reveals with the answer key. Record selection, reveal, retry or skip, feedback, and continuation, reusing ST-NFR-12 timing logs. Confirm both cues remain with reduced motion enabled. Full colour-vision checks belong to Section 4.2.2; questions without wrong options are checked in ST-FR-20.

ST-FR-20: Valid Questions and Build-Time Validation

Type: Functional, Static and Dynamic, automated validation with manual inspection.

Requirements Covered: FR18.

Initial State: Reviewed answer keys and the build validator are available. Prepare separate test content with one, several, or all options correct; no correct answer; and keys naming missing options, both alone and alongside a valid correct option.

Input/Condition: Build with the test content, including a level whose questions are all excluded as invalid. Then build with release content and select each expected correct answer on a separate attempt.

Output/Result: Every build validates its questions. Valid questions remain playable; invalid questions are excluded and logged. Exclusion leaves no empty question screen or blocked level. Each displayed question has at least one correct answer, and every expected correct option is accepted.

Test Case Derivation: FR18 requires valid answers. Build-time validation satisfies SR-17 of the Hazard Analysis [4] and the SRS Phase 3 plan.

How test will be performed: Check that validation runs on every build. Compare its results and packaged question IDs with the reviewed keys; record accepted answers and continuation after exclusions. Reuse validation results for unchanged content, but check the installed build. Clinical approval is covered by ST-NFR-22.

ST-FR-21: Skipping and Incorrect Responses without Penalties

Type: Functional, Dynamic, manual.

Requirements Covered: FR19; related skip, completion, and reward checks under FR8, FR9, FR12, FR31, FR32.

Initial State: For each path below, prepare an unfinished target level with rewards from other levels where available. Record statuses, points, coins, owned accessories, and the target's defined completion reward.

Input/Condition: Run two paths from separate baselines: skip every exercise and animation using the required hold; and make repeated incorrect responses in every applicable exercise and question, then retry or skip. Attempt to skip sensation descriptions, but let them finish. Reach the ending, then repeat each path as a completed-level replay.

Output/Result: All exercises and animations can be skipped except sensation descriptions, which play in full. Skips and mistakes produce no negative text, audio, or visual feedback and remove no points, coins, or accessories. Both paths reach *Completed* with the full defined first-completion reward. Replay preserves completion and gives no duplicate reward.

Test Case Derivation: FR19 prohibits penalties and blocked completion; FR8 and FR9 protects sensation descriptions. FR12, FR31, and FR32 define completion and rewards.

How test will be performed: Compare rewards immediately before and after each skip or incorrect response, then check the completion reward. Add these checks to runs from ST-FR-16, ST-FR-19, and ST-FR-14. Record feedback, each activity's skip availability, final status, and reward totals. Reuse the sensation, clinic-variation, and speak-up message checks from ST-FR-10, ST-FR-11, and ST-FR-12 on those recordings.

ST-FR-22: Repeated Touches during Transitions

Type: Functional, Dynamic, automated input with manual device checks.

Requirements Covered: FR20, FR21.

Initial State: An application screen is ready for navigation. Logging records touch times, accepted actions, and destination display times.

Input/Condition: Trigger each listed transition and send extra touches on the original control, other navigation controls, and a destination control's position. Once the destination appears, send a fresh valid input.

Output/Result: The requested screen appears after at most one navigation action. Extra touches neither skip screens nor execute later on the destination. Fresh input works after the transition.

Test Case Derivation: FR20 and FR21 requires additional touches to be ignored until the next screen appears, including transitions outside levels.

How test will be performed: Use Unity Play Mode tests through the normal input handler. Confirm touch timing, accepted actions, and screen order in the logs; repeat on devices with rapid taps.

ST-FR-23: Narration Protection

Type: Functional, Dynamic, manual.

Requirements Covered: FR20, FR21.

Initial State: Procedure narration is audible. An uninterrupted recording provides its full content and ending.

Input/Condition: Tap, hold, and repeatedly tap outside skip, home, and restart, including other interactive elements. Repeat during mandatory sensation descriptions.

Output/Result: These touches neither skip nor shorten narration.

Test Case Derivation: FR20 and FR21, protects procedure narration from touches outside the three named controls.

How test will be performed: Record touch locations and audio; compare the content and ending with the uninterrupted recording. Home and restart during narration are checked in ST-FR-25 and ST-FR-29.

ST-FR-24: Skip Hold Threshold and Protected Descriptions

Type: Functional, Dynamic, automated input with manual device checks.

Requirements Covered: FR20, FR21, FR8.

Initial State: A skippable activity will remain active throughout the test input, with procedure narration where present. Log touch-down, release, and skip times. Let T be `SKIP_HOLD_MIN_S`.

Input/Condition: On separate attempts, hold skip for $T - 0.1$, T, and $T + 0.1$ seconds. Also make short presses totalling more than T. Repeat during each sensation description. For each input handler, background the app during a short press, release while away, then return and press briefly again with the process retained.

Output/Result: A continuous hold reaching T skips a skippable activity on the next input-processing frame. Shorter holds never skip, and separate presses do not accumulate, including across interruptions. Sensation descriptions cannot be skipped, even with a hold of T or longer.

Test Case Derivation: FR20 and FR21 defines the hold threshold and preserves FR8's exception for sensation descriptions.

How test will be performed: Use timed Unity Play Mode input through the normal handler. Measure actual hold durations from logs; allow the next input-processing frame at T, but no early skip. Confirm short-tap and hold behaviour on devices, and record protected descriptions finishing.

4.1.5 Exiting and Interruptions

These tests verify FR22–FR27. Use the inventory from Section 4.1.4 to check home and restart on every screen in all three levels, including default content and each distinct age-group variant. For FR27, include other application screens with active audio or animation.

Build an interruption matrix covering each available event type on each distinct media or input handler, and each media sequence with at least one applicable event. Reuse runs when the conditions match, and record untested device events as unverified.

ST-FR-25: Return to the Journey Map from Every Screen

Type: Functional, Dynamic, manual.

Requirements Covered: FR22, FR26; one-touch exit from NFR4.

Initial State: Setup is complete and the target level is open. The inventory includes every screen from introduction to ending.

Input/Condition: Select home once on each screen, including during narration, sensation descriptions, and unfinished exercises. Re-enter the level to reach each remaining screen.

Output/Result: Home ends the level and returns to a usable journey map with one touch, without confirmation or waiting for an exercise or narration to finish.

Test Case Derivation: FR22 defines the destination, FR26 requires exit from every screen, and NFR4 sets the one-touch limit.

How test will be performed: Record the starting screen, touch count, and destination for each exit. Reuse these results for NFR4 in Section 4.2.2.

ST-FR-26: Level Audio and Visuals Stop on Exit

Type: Functional, Dynamic, manual with playback logs.

Requirements Covered: FR23.

Initial State: Audio is enabled. The inventory identifies level narration, music, sound effects, animations, and delayed or looping effects.

Input/Condition: Exit before the ending while each media type is active, including overlapping media. Also exit before delayed effects start and wait on the map past their scheduled start or completion.

Output/Result: All level media stop before the map appears and none starts later. The map's own media may play.

Test Case Derivation: FR23 and SR-13 of the Hazard Analysis [4] require level media to stop on exit.

How test will be performed: Reuse exits from ST-FR-25. Record screen and device audio; compare media-stop events with the first map frame in Unity logs. Check for delayed playback and remaining active looping effects.

ST-FR-27: Early Exit Preserves Progress and Rewards

Type: Functional, Dynamic, manual with state inspection.

Requirements Covered: FR24.

Initial State: Prepare a new journey and one with completed levels and earned rewards. After starting the target level, record all level statuses, journey points, coins, and owned accessories, including nonzero rewards where available.

Input/Condition: Exit an unfinished level and a completed-level replay. Include exits after a hint, after a skip, and just before completion.

Output/Result: All recorded values remain unchanged, with no reward issued. The target remains *In Progress* or *Completed* as appropriate; other levels are unaffected.

Test Case Derivation: FR24 leaves player state unchanged on exit. The baseline is taken after starting because FR4 can first change the target to *In Progress*.

How test will be performed: Add before-and-after state snapshots to matching exits from ST-FR-25. Compare recorded values and confirm that exit occurred before the completion event.

ST-FR-28: Start from the Beginning after Exit or Closure

Type: Functional, Dynamic, manual with process logs.

Requirements Covered: FR25.

Initial State: A level is partway through play, with pending saves complete. Record its opening screen and current step.

Input/Condition: On separate runs, leave through home, close the app during play, and terminate it while backgrounded. Reopen the app where needed and start the same level. Cover unfinished levels and completed-level replays.

Output/Result: Each run starts at the level's beginning, without resuming the abandoned step or resetting the journey.

Test Case Derivation: FR25 requires a fresh start after exit or closure; FR27 covers resuming a retained process.

How test will be performed: Reuse exits from ST-FR-27. Confirm process termination for closure cases and compare each start with the recorded opening. Share relaunches with ST-FR-34 to check saved data.

ST-FR-29: Restart from Every Screen without Losing Progress

Type: Functional, Dynamic, manual with state inspection.

Requirements Covered: FR26; restart during narration under FR20, FR21.

Initial State: Prepare an unfinished attempt and a completed-level replay with earned rewards. Record saved statuses and rewards; for ending screens, take the baseline after completion is applied.

Input/Condition: Restart from every listed screen, including narration, sensation descriptions, unfinished actions, hints, reveals, and the ending. Try the first interaction and home after each restart.

Output/Result: Restart returns to the beginning without waiting for narration to finish, with play and home available. Statuses and rewards are unchanged: unfinished levels remain *In Progress*, completed levels remain *Completed*, and restarting gives no reward.

Test Case Derivation: FR26 requires restart from every screen without losing progress; FR20 and FR21 keeps it available during narration.

How test will be performed: Record the starting and returned screens, successful interaction and exit, and before-and-after saved values.

ST-FR-30: Unexpected Inputs Do Not Block Play

Type: Functional, Dynamic, manual.

Requirements Covered: FR26.

Initial State: For each level state, identify inputs with no defined action. Incorrect task responses requiring hints or answer reveals are covered by ST-FR-16 and ST-FR-19.

Input/Condition: Apply unexpected touches and gestures once and repeatedly, including extra simultaneous touches and cancelled drags where undefined. Include unfinished actions. Release the inputs, then try a valid response or permitted skip. On separate attempts, use home and restart before completing the task.

Output/Result: Unexpected inputs are ignored without navigation or changes to saved progress or rewards. Valid input, home, and restart remain usable; mandatory sensation descriptions remain unskippable.

Test Case Derivation: FR26 requires unexpected inputs to be ignored without blocking play; SR-4 of the Hazard Analysis [4] also covers recovery from unfinished actions.

How test will be performed: Record the state, gesture, response, state comparison, and successful continuation or recovery. Reuse ST-FR-22 for transitions and ST-FR-23 for narration.

ST-FR-31: Pause and Resume after Backgrounding or Locking

Type: Functional, Dynamic, manual with playback logs.

Requirements Covered: FR27.

Initial State: Audio is enabled and the process is retained. Record the screen, exercise state, active media positions, level statuses, and rewards. Include narration, music, sound effects, and matching animations.

Input/Condition: Background the app, then return; separately lock and unlock the device. Wait beyond the remaining media or timer duration, or one loop; use the same interval for input-only states. Follow the interruption matrix, including sensation descriptions, unfinished exercises, pending feedback/completion callbacks, and non-level media. Repeat the pause/resume cycle on each distinct handler.

Output/Result: All audio and animation pause without advancing. The same screen and media positions resume with narration and visuals aligned. Timers and callbacks do not consume paused time, skip content on return, or complete the level during the pause. Exercises remain usable, and statuses and rewards are unchanged.

Test Case Derivation: FR27 requires continuation from the paused point; SR-25 of the Hazard Analysis [4] requires media to resume together. Start-offset timing is checked in ST-NFR-14.

How test will be performed: Record device audio and compare media positions at pause with those immediately before playback resumes, within the logging precision. Compare exercise state and rewards, review resumed content, and try valid input, plus home and restart within levels. A crash fails the test; confirmed OS termination is checked under ST-FR-28, followed by another retained-process run.

ST-FR-32: Pause and Resume after Audio or Screen Interruptions

Type: Functional, Dynamic, manual with playback logs.

Requirements Covered: FR27.

Initial State: Use the states from ST-FR-31, with the process retained.

Input/Condition: Trigger available calls, alarms, or notifications that take over audio while the game remains visible, or take over the screen. Dismiss them and return to the game. Include sensation descriptions and exercises awaiting input. Also test a notification that takes over neither audio nor screen, where available.

Output/Result: Actual takeovers meet the pause/resume criteria in ST-FR-31. Notifications that take over neither audio nor screen do not pause play.

Test Case Derivation: FR27 and SR-18/SR-25 of the Hazard Analysis [4] cover interruptions beyond backgrounding. Muting is checked in ST-FR-18.

How test will be performed: Confirm takeover independently of game callbacks, then compare callback and playback logs as in ST-FR-31. Use local alarms or notifications for offline runs; record call-based and simulated checks separately from offline device evidence.

4.1.6 Saving and Restoring Progress

These tests verify FR28, FR29 and FR30 using an offline test installation. Compare the saved level statuses and coins (player state), owned and equipped accessories, and selected character. Journey points and completed-level counts are calculated from that state, as defined by the SRS scoreboard values.

Keep before-and-after snapshots of the complete save data, including any recovery copies. For FR30 cases, ensure no valid fallback save can be loaded. Keep fault-injection hooks out of release builds and repeat normal save and restore checks on release builds. Exclude deliberately delayed or terminated runs from normal performance and reliability measurements.

ST-FR-33: Save Each Change before Processing the Next Input

Type: Functional, Dynamic, automated event checks with manual device checks.

Requirements Covered: FR28, FR29.

Initial State: Prepare valid saves for each change: unstarted levels for start checks, unfinished levels for completion, and sufficient coins and an unowned item for purchases. Use the build's reward and accessory definitions.

Input/Condition: Separately start and first-complete each level; purchase an accessory; equip, replace, and unequip owned items; confirm initial and changed character selections; and confirm a journey reset under FR45. Include accessory rewards where defined. Delay each save and send another valid input through the normal handler. Use an input accepted without the delay, and hold the delay beyond other input locks. Do not navigate away or background the app to trigger saving.

Output/Result: Each complete change is saved before the next input is processed. Purchases save both coin deduction and ownership; completion saves status and rewards. Unaffected values remain unchanged, and unequipping or resetting saves the resulting empty/default values.

Test Case Derivation: FR28 requires saving each change before the next input; FR29 ensures checking only a later launch cannot establish this ordering.

How test will be performed: Log the change, storage commit, and next processed input. Delay only storage, without imposing input ordering. Inspect saved values at commit, not when a write is merely queued. Release the delay and try fresh input; ignored transition touches need not be queued (ST-FR-22). Repeat without delays on devices, reusing starts and completions from ST-FR-13.

ST-FR-34: Restore Saved Progress and Customization across Launches

Type: Functional, Dynamic, manual with process and state logs.

Requirements Covered: FR28, FR29; replay and deletion persistence under FR32, FR45, and FR46.

Initial State: Prepare the mixed statuses from ST-FR-5, coins, owned and equipped accessories, and a selected character. Use separate valid saves if needed, including nonzero balances. Record all values and expected scoreboard totals.

Input/Condition: After each save change in ST-FR-33 commits, terminate and relaunch the app.

Repeat the launch for the mixed-status save and reset state. Replay a restored completed level to its ending. Also confirm clear-all-data and relaunch twice, acknowledging the notice on the first launch.

Output/Result: The latest saved values are restored and agree with the map, scoreboard, shop, and character display. Neither launch nor replay awards duplicate rewards. Reset and clear-all remain effective across launches; reset keeps acknowledgement, while clear-all requires it again.

Test Case Derivation: FR28 and FR29 require restoration; the SRS offline-use case also covers customization. FR32 prohibits repeat rewards, and FR45/FR46 define deletion and acknowledgement.

How test will be performed: Verify process termination and compare loaded and displayed values with the baseline. Include foreground and background closure from ST-FR-28. Add these checks to existing save runs. Apply ST-FR-7 to the restored level and the setup tests to the clear-all notice.

ST-FR-35: Preserve Saved Player State after a Write Failure

Type: Functional, Dynamic, controlled fault injection with state inspection.

Requirements Covered: FR28, FR29.

Initial State: Copy a valid save as the baseline for each distinct save/error-handling path in ST-FR-33. Group actions only when both paths match, and restore the baseline before each run.

Input/Condition: Inject storage-full or write-access failures before writing and failures after writing begins but before commit. Cover each path, including completion, purchase, and reset, with an accessory reward where defined. Keep the fault active until termination to prevent a retry or shutdown save from hiding it; remove the fault before relaunch.

Output/Result: The error causes no crash. Saved statuses and coins remain readable and unchanged, and relaunch restores them without partial changes or a reset. Saving and restoration work once storage is available.

Test Case Derivation: FR28 and FR29 preserves previously saved player state; SR-20 of the Hazard Analysis [4] requires failure handling without crashes.

How test will be performed: Inject errors through a storage hook rather than filling the device. Log the failure point and compare stored and restored values with the baseline. Record customization separately: FR28 does not specify its rollback. Flag inconsistent coin/ownership results for SRS review. After removing the fault, save a new change and relaunch. On a separate run per error path, clear the fault without terminating; verify usable input, including home/restart within levels, then save a new change and relaunch to check restoration.

ST-FR-36: Preserve Saved Player State when a Write Is Interrupted

Type: Functional, Dynamic, controlled process termination with storage logs.

Requirements Covered: FR28, FR29.

Initial State: Use the baselines and save paths from ST-FR-35. Identify write and commit stages.

Input/Condition: Terminate without a shutdown save before writing, after a partial write where applicable, and immediately before commit. Relaunch after each termination, restoring the baseline between runs. Include a control terminated after successful commit.

Output/Result: Interrupted writes restore the previous player state without mixed, damaged, or reset statuses and coins. The completed-write control restores the full new save. Relaunch does not crash.

Test Case Derivation: FR28 and FR29 covers interrupted writes as well as reported failures; termination after commit must retain the new save.

How test will be performed: Use debugger or test hooks to pause at each storage stage, then confirm termination. Compare stored and restored values with the baseline, or the new save for the control. Record customization as in ST-FR-35. Unconfirmed interruption points remain unverified.

ST-FR-37: Recover when Saved Data Is Missing

Type: Functional, Dynamic, manual with storage inspection.

Requirements Covered: FR30.

Initial State: Prepare a fresh installation without restored backups, and a previously used installation

with its progress and customization data removed while closed.

Input/Condition: Launch each installation. Capture recovery in the load log before setup can save data, then acknowledge any required notice without selecting a character or starting a level.

Output/Result: Missing data is replaced without failure. Every level is *Not Started*; coins, journey points, and completed-level count are zero. No accessories are owned/equipped and no character is selected. All levels remain available using the default character.

Test Case Derivation: FR30 specifies new-installation progress and customization; FR34 provides a default character.

How test will be performed: Confirm missing data before launch and inspect the replacement. Reuse ST-FR-4 for map checks, then start a level, wait for saving, and relaunch to confirm new progress persists. Retain snapshots and load logs.

ST-FR-38: Recover when Saved Data Cannot Be Read

Type: Functional, Dynamic, controlled fault injection with state inspection.

Requirements Covered: FR30.

Initial State: Valid saved progress and customization exist and have been copied. The app is closed; storage remains writable.

Input/Condition: Launch with a hook causing an access or I/O error when reading the existing save, while allowing replacement data to be written.

Output/Result: The app replaces unreadable data with the defaults in ST-FR-37 without failure or partial recovery. Later launches use the replacement rather than restoring the old data.

Test Case Derivation: FR30 separately requires recovery from existing data that cannot be read.

How test will be performed: Confirm the save exists and the read error reaches the load path. Inspect replacement data, remove the fault, and relaunch. Repeat the continued-play/save check in ST-FR-37, distinguishing injected errors from reproduced device failures.

ST-FR-39: Recover when Saved Data Is Invalid

Type: Functional, Dynamic, prepared invalid data with device checks.

Requirements Covered: FR30.

Initial State: Copy a valid save with progress and customization. Identify required fields and allowed empty values in the storage format. Change one condition per invalid copy, leaving other fields valid.

Input/Condition: Launch separately with empty, malformed, or truncated data; missing required fields; wrong types; negative coins; unknown statuses or character/accessory IDs; missing, unknown, or duplicate level records where representable; unowned equipped items; two equipped items in one category; and a reward-claimed flag inconsistent with completion status if stored. Include an earlier-format save only if current validation and any supported migration reject it.

Output/Result: Validation rejects invalid data before gameplay or progress display. The whole progress/customization state is replaced with the defaults in ST-FR-37, without failure or retained old values. Replacement data remains usable across launches.

Test Case Derivation: FR30 requires replacement; SR-20 of the Hazard Analysis [4] requires validation before use. Valid values follow the player-state model, data dictionary, and FR39.

How test will be performed: Restore the baseline for each case. Record the mutation and rejection in load logs; compare replacement data and displays with the defaults, then repeat the continued-play/save check in ST-FR-37. Use valid populated and new-player saves as controls, including unset optional fields, empty inventories, and completed levels with zero coins after spending where reachable. Valid controls must load without recovery.

4.1.7 Rewards and Scoreboard

These tests verify the level rewards and the personal scoreboard in FR31–FR33 of the SRS [1]. They use the test levels and saved states in Appendix 6.2, so the expected values do not change when the real rewards are tuned. Each test is an automated Unity Play Mode test that performs the actions and then checks the saved game state.

ST-FR-40: Coin Reward on First Completion

Type: Functional, Dynamic, automatic.

Requirements Covered: FR31.

Initial State: The saved state is `SAVE_NEW`.

Input/Condition: Start `L1`, skip its exercise, and answer its question incorrectly before selecting the correct answer. Reach the end of the level.

Output/Result: The reward screen shows 10 journey points and 5 coins earned. The level is Completed, the coin balance is 5, and the total journey points are 10.

Test Case Derivation: FR31 gives the level's journey points and coins when it is finished for the first time. The reward depends on finishing the level, not on the answers, so skipping and a wrong answer do not reduce it.

How test will be performed: The Play Mode test plays the level, reads the values on the reward screen, and then checks the saved level status, coin balance, and total journey points.

ST-FR-41: Accessory Reward on First Completion

Type: Functional, Dynamic, automatic.

Requirements Covered: FR31.

Initial State: The saved state is `SAVE_NEW`.

Input/Condition: Complete `L3`. Do not equip the accessory that is offered.

Output/Result: The reward screen shows 40 journey points, 15 coins, and the Sun hat, with an option to equip it. The Sun hat is owned but not equipped. The coin balance is 15.

Test Case Derivation: FR31 includes the level's accessory in the reward when its Reward Definition has one. UC9 offers to equip the accessory but does not equip it.

How test will be performed: The Play Mode test completes the level, reads the reward screen, and then checks the saved coin balance and the owned and equipped accessories.

ST-FR-42: No Duplicate Reward on Replay

Type: Functional, Dynamic, automatic.

Requirements Covered: FR32.

Initial State: The saved state is `SAVE_MIXED`.

Input/Condition: Replay `L3` to its end `REPLAY_TRIALS_MIN` times. Restart the game session before the last replay.

Output/Result: Each replay ends with a congratulation and no points, coins, or accessory. The total journey points stay at 50, the coin balance stays at 3, and the owned accessories do not change.

Test Case Derivation: FR32 forbids a second reward for a level that has already been rewarded. The coins from `L3` were already spent in `SAVE_MIXED`, so a duplicate reward would raise the balance to 18.

How test will be performed: The Play Mode test replays the level and checks the end-of-level screen and saved game state after each replay. It restarts the session by reloading the game from the save file.

ST-FR-43: Scoreboard for a New Player

Type: Functional, Dynamic, automatic.

Requirements Covered: FR33.

Initial State: The saved state is `SAVE_NEW`.

Input/Condition: Open the scoreboard from the journey map.

Output/Result: Each test level is shown as Not Started with 0 points. The scoreboard shows 0 total journey points, 0 of 3 levels completed, and 0 coins. No nickname is shown, and the wording is not personalized.

Test Case Derivation: A new player has no completed levels and no coins, so every scoreboard value is 0. FR43 requires wording that is not personalized when no nickname is set.

How test will be performed: The Play Mode test opens the scoreboard and checks each value shown against the expected result.

ST-FR-44: Scoreboard with Progress

Type: Functional, Dynamic, automatic.

Requirements Covered: FR33.

Initial State: The saved state is `SAVE_MIXED`.

Input/Condition: Open the scoreboard, return to the journey map, and open it again.

Output/Result: `L1` is Completed with 10 points, `L2` is In Progress with 0 points, and `L3` is Completed with 40 points. The scoreboard shows 50 total journey points, 2 of 3 levels completed, 3 coins, the nickname Sam, and Test character B. It shows no rank or results from other players. Viewing it does not change the saved game state.

Test Case Derivation: FR33 counts points only for completed levels, so the total is $10 + 40 = 50$. The coin balance is stored separately and is 3 after the Blue scarf was bought. The scoreboard must not compare the child with other users.

How test will be performed: The Play Mode test opens the scoreboard twice and checks each value shown. It compares the saved game state before and after viewing the scoreboard.

4.1.8 Characters and Accessories

These tests verify the character selection and customization features defined in the SRS. They cover how players interact with available characters and accessories and how their selections affect the game.

ST-FR-45: Default to Character Selection and Diversity Inventory

Type: Functional, Dynamic, manual.

Requirements Covered: FR34, FR35.

Initial State: No selected character, then a test character with known progress and items.

Input/Condition: Enter character selection, preview and cancel, then confirm a different character. Inspect every character/accessory catalogue entry against FR35 attributes.

Output/Result: The unselected state uses a default character and does not block play. Every selectable character is shown. A preview/cancel does not commit a selection but confirmation changes only the selected character, preserving progress, coins, owned and equipped items. The catalogue includes variation in skin tone, hair, body type, gender expression, accessibility/medical aids, and cultural/religious accessories as required.

Test Case Derivation: FR34 defines selection-only state changes and default behaviour for the character selection. FR35 defines the attributes for the diversity inventory.

How test will be performed: Save the full state before and after the preview and confirmation steps, and compare them. Restart the app to check that the selection is remembered. Keep the catalogue checklist and any open issues about how characters are represented in inventory.

ST-FR-46: Purchase Flow with Previews, Affordability, and Ownership with No Real-Money

Type: Functional, Dynamic, manual.

Requirements Covered: FR36, FR37, FR38, FR40.

Initial State: An unowned fixture item at a fixed price in coins, an already-owned item, and separate balances of price-minus-one coin, exact price, and price-plus-one coin.

Input/Condition: Preview/cancel each item, confirm affordable purchases, attempt an unaffordable purchase, and attempt to buy an owned item. Inspect every purchase path offline and online.

Output/Result: The shop shows every priced item's preview on the selected character, price, and ownership. Preview/cancel changes nothing. Exact-price purchase leaves zero coins and adds the item once. Price-plus-one leaves one coin. Price-minus-one is refused reports one more coin needed. The owned item purchase is refused unchanged and offers equip. No real-money offer, payment flow, or change to journey points/completion exists.

Test Case Derivation: FR36 requires that preview contains character, price, and ownership information and FR40 ensures real-money purchases are excluded from the preview and purchase flow. FR37 and FR38 distinguishes the boundary cases of purchases based on price by sufficient and insufficient balance/ownership status respectively.

How test will be performed: Repeat every purchase path offline and online: set up the fixture with one unowned item at fixed price in coins, one owned item, and three separate balances: price minus one, exact price, and price plus one and save the full state before each step. For each item, open the preview and cancel, then confirm a purchase at the exact-price, price-plus-one balances, and at the price-minus-one balance. For each state, check the expected coins remains (0 or 1) and check that it is refused with a message saying one more coin is needed. Attempt to buy the owned item and check that it is refused with an equip option is offered. After each step, compare the before state with the current state, ensuring no real-money offer appears, and that journey points/completion are unchanged.

ST-FR-47: Equip, Unequip, and Ownership Restrictions of Accessories

Type: Functional, Dynamic, manual.

Requirements Covered: FR39

Initial State: Two owned hats A and B, one owned accessory C from another category, one unowned accessory D, and known balances and progress.

Input/Condition: Equip A, then B, then C. Unequip B. Attempt to equip D, and preview and cancel D. Relaunch after the equipment changes have completed to verify the updated state and that balances and progress are unchanged.

Output/Result: B replaces A in the hat category, and A stays owned but unequipped. C remains equipped in its own category. Unequipping B removes only the equipped association, leaving no hat equipped. D cannot be equipped, and preview/cancel commits nothing. All owned accessories remain owned, and balance, progress, and character selection are unchanged. The updated equipment status is saved and restored on relaunch.

Test Case Derivation: FR39 permits equipping and unequipping only owned accessories, with at most one per category, without deleting ownership.

How test will be performed: Perform on offline and online states: compare the category inventory of accessories, the ownership sets of accessories, and the rendered character before and after each step. After the final step, relaunch the app and compare the restored state with the saved state.

4.1.9 Caregiver Features

These tests verify the caregiver-facing features defined in the SRS. They cover access to caregiver information, application preferences, and other caregiver controls, ensuring that the available functionality behaves as specified.

ST-FR-48: Caregiver Settings and Guides with Gate, Nickname, and Preferences

Type: Functional, Dynamic, manual.

Requirements Covered: FR1, FR15, FR41, FR42, FR43, FR44

Initial State: Acknowledged profile with known state and caregiver access closed.

Input/Condition: Attempt to open every settings entry without completing the written gate, then enter an incorrect then correct written response. Open each procedure guide and the medical notice for viewing. Set nickname “Eye”, change to “Iris”, cancel an edit, then clear it. Change age, narration/music/effect volume, visual cue size, and reduced motion while settings stay open.

Output/Result: No settings or reset control bypasses the reading-based gate. The correct response opens settings. Each guide explains the child’s experience and the matching level, and the medical notice is presented. The nickname is optional, editable, and clearable, and cancelled edits are not saved. All listed preference changes take effect immediately without a restart, and essential visual cues remain available. No request for a real name or health record appears.

Test Case Derivation: FR41, FR42, FR43, FR44 define the features and settings offered to caregivers; FR1 stores medical notice acknowledgment in the procedure guide and FR15 ensures settings are changed only within caregiver settings. No undocumented nickname length or clinical-data input is

introduced as an acceptance rule.

How test will be performed: List every route to settings and try each one before passing the gate, confirming it is blocked. Open each procedure guide and the medical notice and check their content. After each nickname edit, compare the saved state with the previous one. After each preference change, check its effect immediately, without restarting the app. Once all changes are finished, relaunch the app and verify that the saved preferences are restored.

ST-FR-49: Journey Reset and Clearing All Data

Type: Functional, Dynamic, manual.

Requirements Covered: FR1, FR2, FR43, FR45, FR46

Initial State: Set up a test profile with the medical notice already acknowledged and these non-default settings: a different age group, changed narration, music, and effect volumes, larger visual cues, and reduced motion. Repeat with a profile that has an empty journey. Record every preference before deleting anything.

Input/Condition: Start a journey reset and cancel it, then start it again and confirm. Restore the fully populated test profile, then start a clear-all and cancel it, then start it again and confirm. After each confirmed reset or clear-all, restart the app and check that the built-in content is still there.

Output/Result: Each request explains the deletion and asks confirmation, with cancellation changing nothing. Confirmed journey reset returns every level to NotStarted, points/coins to zero, removes owned/equipped items, deselects the character, and clears nickname. Medical notice acknowledgement and the other caregiver preferences (age, audio volumes, cue size, and reduced motion) are retained and home appears. Confirmed clear-all additionally removes medical notice acknowledgement and all local preferences, restoring installation defaults and showing the notice before home. Both preserve packaged levels/assets and their offline availability. An empty journey can also be cancelled or reset safely.

Test Case Derivation: FR45 distinguishes a journey reset from FR46 clear-all. The clear-all reset creates the new player state ensuring by FR1 that the medical notice acknowledgement is presented. The journey reset retains the medical notice acknowledgement in caregiver settings by FR2.

How test will be performed: Save a copy of everything the app stores on the device before and after each case (journey reset and clear-all, each cancelled and confirmed). After each confirmed case, restart the app and compare the stored data with a freshly installed app. Check that all levels can still be played offline and that no deleted nickname or inventory item reappears.

4.2 Tests for Nonfunctional Requirements

The tests in this section verify the non-functional requirements (NFRs) defined in the SRS. These requirements describe how well the game should operate, rather than the specific features it provides. The tests are organized according to the NFR categories in the SRS, with each test identifying the requirement it covers, the conditions for testing, the expected result, and how the test will be performed. Test parameters are listed in Section 6.1.

For the Capstone course, the team will test on the limited physical devices available and use Unity tools to simulate additional device configurations where appropriate. The devices or simulations used will be recorded when the tests are performed. Any additional changes to the testing scope for the Capstone course will be explained in a *Remark* under the relevant test.

4.2.1 Look and Feel Requirements

These tests verify the control layout and the content safety requirements in NFR1–NFR2 of the SRS [1]. The survey in Appendix 6.4 adds participant feedback on both requirements.

ST-NFR-1: Consistent Control Positions

Type: Non-functional, Dynamic, automatic.

Requirements Covered: NFR1.

Initial State: The latest build is open in the Unity editor. The designated position of each shared

control, such as the exit, hint, and settings buttons, is recorded in the design.

Input/Condition: Visit every screen of every level, the journey map, the scoreboard, and the character screen at each test screen size.

Output/Result: On every screen where a shared control appears, its centre is within `CONTROL_POSITION_TOLERANCE_MM` millimetres of its designated position.

How test will be performed: A Play Mode test opens each screen and records the on-screen position of each shared control. It converts the distance from the designated position to millimetres using the screen density of each test screen size. A team member reviews any screen that fails and checks the results on one physical device.

ST-NFR-2: Child-Safe Content Review

Type: Non-functional, Static content inspection.

Requirements Covered: NFR2.

Review procedure: One team member will prepare a list of every image, animation, narration clip, and sound in the release build, with a recording of each level. A child-life specialist will review each item and answer two questions. Is the item frightening for a child aged 3 to 7? Does it show pain or distress beyond the sensations in the approved script? A second team member will record each finding in GitHub Issues. Changed content will be sent back for review.

Expected result and evidence: Every item on the list has been reviewed, and no finding is unresolved at release. The team will keep the content list, the completed checklist, and the related GitHub Issues.

Remark: If a child-life specialist is not available during the capstone, the project supervisor will review the content instead. This would not fully verify the requirement, since the SRS names a child-life specialist as the reviewer.

4.2.2 Usability and Humanity Requirements

These tests verify the usability and humanity requirements in NFR3–NFR10 of the SRS [1]. Children and caregivers will not take part in testing during the capstone (Section 2.2). The tests that need users will use classmates and clinical staff instead, as explained in the remark under each test. The survey in Appendix 6.4 adds participant feedback on these requirements.

ST-NFR-3: Touch Target Size

Type: Non-functional, Dynamic, automatic with manual check.

Requirements Covered: NFR3.

Initial State: The latest build is open in the Unity editor and installed on the test devices.

Input/Condition: Visit every screen of the game at each test screen size.

Output/Result: Every interactive element has a touch target at least `TOUCH_TARGET_MIN` millimetres wide and high.

How test will be performed: A Play Mode test opens each screen, finds every interactive element, and converts the size of its touch area to millimetres using the screen density. A team member then measures the smallest elements on each physical test device with a ruler or a screenshot of known scale.

ST-NFR-4: One-Touch Exit from Every Screen

Type: Non-functional, Dynamic, automatic.

Requirements Covered: NFR4.

Initial State: The latest build is open in the Unity editor.

Input/Condition: Open each level, and on every screen of the level, touch the exit button once.

Output/Result: On every screen of every level, one touch shows the journey map.

How test will be performed: A Play Mode test steps through each level screen by screen. On each

screen, it touches the exit button once and checks that the journey map is shown. It then reopens the level and continues from the next screen.

ST-NFR-5: Core Tasks Without Instructions

Type: Non-functional, Dynamic, manual.

Requirements Covered: NFR5.

Initial State: The latest build is installed on a test device. The participant has not used the game before and has not been shown how it works.

Input/Condition: At least `USABILITY_PARTICIPANTS_MIN` participants each complete the three core tasks. They start a named level from the journey map, play a level to its end, and leave a level early. Each participant acts as one of the child personas in the SRS.

Output/Result: At least `UNAIDED_RATE_MIN`% of participants complete each task without verbal help. The median time for each task is at most `CORE_TASK_MAX_MIN` minutes.

How test will be performed: A team member gives each task in one sentence and then does not help. They record whether the task was completed without help and how long it took. They also note where the participant hesitated or made a wrong touch. The participant then completes the survey in Appendix 6.4.

Remark: The participants will be classmates rather than children, because the capstone does not include testing with children. This finds usability problems but does not verify the requirement for the intended users.

ST-NFR-6: Clinician Finds a Named Level

Type: Non-functional, Dynamic, manual.

Requirements Covered: NFR6.

Initial State: The latest build is installed on a test device and is open on the home screen. The participant is a clinician or clinical staff member who has not used the game before.

Input/Condition: A team member trains the participant for at most `CLINICIAN_TRAINING_MAX_MIN` minutes. The participant is then handed the device and asked to open the introduction of a named level.

Output/Result: Every participant opens the introduction within `CLINICIAN_FIND_MAX_MIN` minute of picking up the device.

How test will be performed: A team member times the training and the task, and names a different level for each participant. They record the time taken and any wrong touches. The participant then completes the survey in Appendix 6.4.

Remark: The number of participants depends on the clinical staff the project supervisor can arrange. If only the supervisor takes part, the result will be recorded as preliminary.

ST-NFR-7: No Pressure to Keep Playing

Type: Non-functional, Static design and build inspection.

Requirements Covered: NFR7.

Review procedure: Two team members will separately go through every screen of the release build with a checklist of the patterns in the requirement. These are rewards for consecutive days, rewards or offers that expire, countdown timers outside an exercise, randomized rewards, and notifications. They will also check that the Android manifest and iOS project settings do not request permission to send notifications. Any pattern found will be recorded in GitHub Issues and checked again after it is fixed.

Expected result and evidence: The checklist finds none of the patterns, and neither build requests notification permission. The team will keep the completed checklists and the build settings checked.

ST-NFR-8: Sentence Length and Reading Level

Type: Non-functional, Static text inspection, automatic with manual review.

Requirements Covered: NFR8.

Review procedure: A script will count the words in every narrated sentence of the narration scripts. It will also compute the Flesch–Kincaid grade of each text in the caregiver area. A child-life specialist will then read the narration scripts and list any word a child aged 3 to 7 may not know. Each sentence or word that fails will be rewritten and checked again.

Expected result and evidence: No narrated sentence has more than `SENTENCE_MAX_WORDS` words, no unfamiliar word is unresolved, and every caregiver-area text has a grade of at most `ADULT_TEXT_MAX_GRADE`. The team will keep the script output and the specialist's word list.

Remark: If a child-life specialist is not available, the project supervisor will review the words instead, as in ST-NFR-2.

ST-NFR-9: WCAG 2.2 Level AA Audit

Type: Non-functional, Static accessibility audit with manual in-app checks.

Requirements Covered: NFR9.

Review procedure: One team member will list the WCAG 2.2 Level AA success criteria that apply to a native mobile game, with a reason for each criterion left out. They will check each criterion on the test devices, using a contrast checker for text and controls and the screen reader for labels. A second team member will repeat the audit separately. Each failure will be recorded in GitHub Issues and checked again after it is fixed.

Expected result and evidence: The audit finds no failure of an applicable criterion. The team will keep the list of criteria, both completed audits, and the related GitHub Issues.

ST-NFR-10: Screens Without Colour

Type: Non-functional, Dynamic, manual with simulation.

Requirements Covered: NFR10.

Initial State: The latest build is open in the Unity editor with a colour vision deficiency filter.

Input/Condition: View every screen under protanopia, deuteranopia, and tritanopia simulation. Include the journey map with all three level statuses and a revealed correct answer in each level.

Output/Result: Every state and choice can be told apart by shape, position, or text as well as colour. Every revealed answer has at least two cues that do not rely on colour, and both are visible in all three simulations.

How test will be performed: A team member takes a screenshot of each screen under each simulation. A second team member reviews the screenshots and records any state or choice that can only be told apart by colour.

4.2.3 Performance Requirements

These tests verify the performance requirements defined in the SRS. They cover startup time, responsiveness, animation and audio behaviour, reliability, and resource usage during normal gameplay.

ST-NFR-11: Startup Time

Type: Non-functional, Dynamic, manual.

Requirements Covered: NFR11.

Initial State: The latest build is installed on the test devices, and the app is closed.

Input/Condition: Open the app from its icon at least `STARTUP_TRIALS_MIN` times on each test device. Close the app fully between launches.

Output/Result: The median time to reach the usable home screen is at most `STARTUP_MAX_S` seconds on each device tested.

How test will be performed: Record each launch and measure the time from tapping the icon to when the home screen is ready. Calculate the median for each device.

ST-NFR-12: Touch Response

Type: Non-functional, Dynamic, manual with timing logs.

Requirements Covered: NFR12.

Initial State: The app is open on the test devices.

Input/Condition: Make at least `TOUCH_TRIALS_MIN` valid touches across the menus and games. Also test each type of incorrect answer and time the correct-answer reveal.

Output/Result: On each device tested, at least `RESPONSE_RATE_MIN`% of touches show a response within `RESPONSE_MAX_MS` milliseconds. Every correct-answer reveal takes at most `ANSWER_REVEAL_MAX_S` second.

How test will be performed: Use Unity timestamps to record each touch event and its first visible response. Calculate the percentage that meets the limit, and record the time taken to reveal the correct answer after an incorrect response.

ST-NFR-13: Animation Frame Rate

Type: Non-functional, Dynamic, automatic.

Requirements Covered: NFR13.

Initial State: The latest build is installed on the test devices.

Input/Condition: Play through each procedure walkthrough and practice game while recording frame times.

Output/Result: On each device tested, the game runs at or above `FRAME_RATE_MIN` frames per second for at least `FRAME_RATE_COMPLIANCE_MIN`% of the measured time.

How test will be performed: Use the Unity Profiler to collect frame times throughout each walkthrough and game. Calculate the percentage of measured time that meets the required frame rate.

ST-NFR-14: Narration and Animation Timing

Type: Non-functional, Dynamic, automatic with manual review.

Requirements Covered: NFR14.

Initial State: The latest narration and animations are included in the build.

Input/Condition: Play each narrated step in every procedure walkthrough on the test devices.

Output/Result: The narration starts within `AV_SYNC_MAX_MS` milliseconds of its matching animation for every step.

How test will be performed: Add temporary Unity logs at the start of each narration clip and its animation. Compare the timestamps and review a recording of any step that appears out of sync.

ST-NFR-15: Audio Levels

Type: Non-functional, Static audio inspection.

Requirements Covered: NFR15.

Review procedure: One team member will export the final narration, sound effects, and music with the game's audio mixer settings applied, then compare their peak levels using Audacity or a similar audio editor. A second team member will check the measurements and mixer settings. Any sound above the narration peak will be adjusted and checked again.

Expected result and evidence: No sound effect or music has a peak level above narration. The team will save the measurement results, mixer settings, and any issues corrected.

ST-NFR-16: Flashing Content

Type: Non-functional, Dynamic, automated analysis with manual review.

Requirements Covered: NFR16.

Initial State: All animations, transitions, and reward effects are included in the build.

Input/Condition: Record or export every animation and visual effect, including repeated or looping effects.

Output/Result: No sequence contains more than `FLASH_MAX_PER_S` flashes in any one-second period.

How test will be performed: A team member will run the recordings through a photosensitive-epilepsy flash-analysis tool. A second team member will review any flagged sections. Save the results and rerun the analysis after fixing any failing effects.

ST-NFR-17: Crash-Free Sessions

Type: Non-functional, Dynamic, manual.

Requirements Covered: NFR17.

Initial State: The current build is installed on the test devices and crash logs are available.

Input/Condition: Complete at least `TEST_SESSIONS_MIN` sessions across the test devices. Include normal gameplay, restarting levels, saving, closing and reopening, and suspend/resume.

Output/Result: At least `CRASH_FREE_RATE_MIN`% of sessions finish without a crash or freeze.

How test will be performed: Record the result of each session. Check crash logs and count the app as frozen if it does not respond to a normal action for `FREEZE_OBSERVATION_S` seconds. Calculate the percentage of sessions completed without a crash or freeze.

ST-NFR-18: CPU Usage During Gameplay

Type: Non-functional, Dynamic, automatic.

Requirements Covered: NFR18.

Initial State: The app is installed on the test devices.

Input/Condition: Play continuously for at least `HEAT_TEST_SESSION_MIN` minutes on each test device, moving between the different activities.

Output/Result: Measured CPU utilization stays at or below `CPU_UTILIZATION_MAX_PERCENT`% throughout the session on each device tested.

How test will be performed: Use the Unity Profiler and available device monitoring tools to record CPU utilization during gameplay. Record the highest measured value and any thermal warning.

4.2.4 Operational and Environmental Requirements

These tests verify the operational and environmental requirements defined in the SRS. They check that the game can be installed and used offline, meets the storage limit, and includes clinically approved content.

ST-NFR-19: Offline Functionality

Type: Non-functional, Dynamic, manual.

Requirements Covered: NFR19.

Initial State: The game is installed with all content, but has not been opened for the first time.

Input/Condition: Turn on airplane mode and disable Wi-Fi. Run all functional tests without connecting to the internet.

Output/Result: Every functional test passes without requiring a network connection.

How test will be performed: Run the full set of functional tests on the test devices and record the results. Record any missing content, error message, or feature that cannot be used offline.

ST-NFR-20: Installation and First Launch

Type: Non-functional, Dynamic, manual.

Requirements Covered: NFR20.

Initial State: The app is not installed on the test device.

Input/Condition: Install the latest release build and open the app for the first time.

Output/Result: The home screen can be reached without creating an account, granting permissions, or downloading more content.

How test will be performed: Install the build on each test device, record the first launch, and note

any prompt or extra step required before reaching the home screen.

Remark: For this capstone, the build will be installed directly for testing rather than through a public app-store listing. The store-installation part of the SRS requirement will remain unverified unless the app is published.

ST-NFR-21: Installed Application Size

Type: Non-functional, Static build inspection.

Requirements Covered: NFR21.

Review procedure: Once the iOS and Android release builds are ready, the team member responsible for each build will check the installed application size using the platform's storage information. A second team member will check the reported numbers against `INSTALL_SIZE_MAX_MB` MB. The build version, measured size, and source of the measurement will be recorded.

Expected result and evidence: The installed size of each build is no more than `INSTALL_SIZE_MAX_MB` MB. The team will keep screenshots or build reports showing the size.

Remark: The SRS fit criterion uses the size reported by each app store. For the capstone, the team will use locally measured installed size unless a store release is available, and record that difference in the results.

ST-NFR-22: Clinical Review of Game Content

Type: Non-functional, Manual document review.

Requirements Covered: NFR22.

Review procedure: One team member will prepare the latest game walkthroughs, narration, and procedure guide, with the version of each item recorded. A paediatric ophthalmologist will review the content against the procedures used at the clinic. Another team member will record the feedback and any required changes. Updated content will be sent back for approval.

Expected result and evidence: The ophthalmologist approves every level and procedure-guide entry for the version being released, with no unresolved concerns. The team will keep the content checklist and written approval.

4.2.5 Maintainability and Support Requirements

These tests verify the maintainability and support requirements defined in the SRS. They cover the effort needed to make likely changes, compatibility with supported devices, and testing after third-party dependency updates.

ST-NFR-23: Effort to Make a Likely Change

Type: Non-functional, Manual estimation review.

Requirements Covered: NFR23.

Review procedure: Once the main game is implemented, each team member will estimate how many hours it would take to implement and test each likely change listed in the SRS, excluding clinical review. The team will discuss the estimates and agree on a final number for each change. These will be compared with the original development effort.

Expected result and evidence: Each estimated change effort is less than `MAINT_FRACTION` of the original development effort. The team will keep the individual estimates, agreed estimates, and comparisons.

Remark: For the scope of the capstone, the team will estimate the effort rather than carry out every likely change. This gives an indication of maintainability but does not fully verify the SRS criterion based on recorded effort.

ST-NFR-24: iOS and Android Compatibility

Type: Non-functional, Dynamic, manual.

Requirements Covered: NFR24.

Initial State: The latest build is ready and the test configurations have been recorded.

Input/Condition: Run all functional tests on the supported test configurations and check the game's layouts and controls at screen sizes from `SCREEN_MIN_IN` to `SCREEN_MAX_IN` inches.

Output/Result: All functional tests pass on the configurations tested, and the layouts and controls remain usable at the tested screen sizes.

How test will be performed: Run the tests and record the device or configuration, OS version, screen size, and results. List any supported iOS or Android versions or screen sizes that could not be checked.

ST-NFR-25: Third-Party Dependency Checks

Type: Non-functional, Dynamic regression testing with dependency inspection.

Requirements Covered: NFR25.

Initial State: The Unity project, dependency files, and release build are available.

Input/Condition: Check the Unity version and all third-party package and plugin versions. Run all functional tests on the release build after each dependency change.

Output/Result: Every dependency has an exact version, and all functional tests pass after each dependency change with no unresolved issue before release.

How test will be performed: One team member will prepare the dependency list from the Unity Package Manager and package files. A second team member will check the versions against the project. After a version change, the team will run the functional tests and keep the results with the dependency list.

4.2.6 Security Requirements

These tests verify the security requirements defined in the SRS. They check how the game stores and protects local data, whether it collects only the data it needs, and whether its dependencies have known security issues.

ST-NFR-26: Private Local Storage

Type: Non-functional, Dynamic, manual.

Requirements Covered: NFR26.

Initial State: A test nickname, settings, and game progress have been saved on an unmodified test device.

Input/Condition: Try accessing the saved files from the device file manager, a different app, and normal sharing options.

Output/Result: The saved data cannot be read by another app or through device file-sharing features.

How test will be performed: Confirm where Unity writes the saved data and inspect the app's storage and sharing settings. Record the access attempts and check that the game can still load its own saves.

ST-NFR-27: Release Signing and Distribution

Type: Non-functional, Static release inspection.

Requirements Covered: NFR27.

Review procedure: If the app is prepared for public release, the team members responsible for the iOS and Android builds will check the signatures using Xcode and Android build tools. Another team member will review the signing results and confirm that the planned public distribution channels are the Apple App Store and Google Play Store. The team will save the signing reports and store-release records, if available.

Expected result and evidence: Both release builds have valid signatures, and public distribution uses only the official stores.

Remark: This test will likely be omitted during the capstone because publishing the app to the Apple App Store and Google Play Store is difficult within the course timeline. The team may check signing on test builds, but that would not verify the full release and distribution requirement.

ST-NFR-28: Network Activity

Type: Non-functional, Dynamic, manual with network capture.

Requirements Covered: NFR28.

Initial State: The game is installed on a test device connected to a monitored Wi-Fi network.

Input/Condition: Capture network traffic while launching the game, playing each level, changing settings, earning rewards, and closing and reopening the game.

Output/Result: The game makes no outgoing network requests.

How test will be performed: Capture traffic using a network monitor or test proxy and look for requests associated with the app. Also check the Unity project for enabled analytics or network services. Keep the capture and record any unexplained request.

ST-NFR-29: Stored Data Fields

Type: Non-functional, Dynamic save-file inspection.

Requirements Covered: NFR29.

Initial State: The app is installed with a fresh save file.

Input/Condition: Set a test nickname, complete activities, earn and spend coins, and change the available settings. Then inspect the stored files.

Output/Result: The app stores only fields belonging to Local Preferences, Character Inventory, Game Progress, and Journey Progress in the SRS data dictionary.

How test will be performed: One team member will read the saved data using a development build or Unity editor. A second team member will compare the fields against the SRS data dictionary and record any extra fields.

ST-NFR-30: Dependency Vulnerability Check

Type: Non-functional, Static dependency inspection.

Requirements Covered: NFR30.

Review procedure: One team member will list the exact versions of Unity, packages, and plugins included in the release. The member will check these versions against known-vulnerability databases and supplier advisories. A second team member will review any high or critical findings and check that fixes or safe versions have been applied.

Expected result and evidence: No included third-party library has a publicly known high or critical vulnerability at the time of release. The team will keep the dependency list, the dated scan or advisory results, and any related GitHub issues.

4.2.7 Cultural and Political Requirements

The following reviews verify the inclusivity requirement in the SRS [1] and address representation concerns identified in the Hazard Analysis [4] through static inspections by team members rather than usability studies. Reviewers will examine all released written and visual content, recording each item's file, screen, or asset ID and build version.

ST-NFR-31: Inclusivity of Written and Spoken Content

Type: Non-functional, Static content inspection.

Requirements Covered: NFR31.

Review procedure:

The assigned reviewer and a second team member will separately review all written and spoken content in the final game, including instructions, narration, and caregiver guides, using a checklist of the game's content. For each item, reviewers will answer the following questions:

1. Does the content assume that every child has a mother and father or a particular family arrangement?
2. Does the content assign abilities, interests, or roles based on gender?

3. Does the content assume that the user belongs to a particular culture, religion, or ethnicity?
4. Does the content imply that one cultural background or family arrangement is normal while others are not?
5. Does the content contain any stereotypes, exclusionary terms, or potentially discriminatory language?

For each question, reviewers will record a Yes/No response. Any "Yes" answer will require a finding containing the content ID, wording, concern, and proposed change. Reviewers will compare findings, record their resolution in GitHub Issues, and reinspect corrected content before release.

Expected result and evidence: Every inventory item has a completed five-question checklist, and there are zero unresolved inclusivity findings for the reviewed release. The saved checklist, content inventory, reviewers' names, build version, and GitHub Issue references provide evidence of the review.

ST-NFR-32: Inclusivity of Images, Characters, and Rewards

Type: Non-functional, Static visual inspection with manual in-app confirmation.

Requirements Covered: NFR31.

Initial State: The final build and complete visual asset inventory are available; initial setup has been completed.

Input/Condition: Inspect every level illustration, introductory or instructional animation, journey-map image, selectable character, and cosmetic accessory. Open each available character and accessory in the app to confirm how it appears in context.

Output/Result: No image or character presentation assumes that all children share one family structure, culture, religion, gender, or ethnicity. No visual reinforces a stereotype or depicts a particular identity as the only valid or normal one. All identified issues are resolved before release.

Review procedure:

The assigned reviewer will capture or export the release visuals and record each asset ID. A second team member will independently inspect the same visual asset inventory. Both reviewers will answer the following questions for each asset:

1. Does the image or character depict one family structure, gender, cultural identity, or religious identity as universal?
2. Does the image or character imply that a person's identity determines their abilities, interests, or role?
3. Are cultural or religious items represented in a misleading, inaccurate, or stereotypical manner?
4. Does the released character library contain representations that conflict with the inclusivity intent described in the Hazard Analysis?

For each question, reviewers will record a Yes/No response and explanatory notes. Any "Yes" answer will be documented as a potential concern. The team will track findings in GitHub Issues, correct the affected assets, and repeat the inspection to verify that the concerns have been addressed.

Expected result and evidence: A complete visual checklist, screenshots or exported assets, and an issue-resolution log demonstrate zero unresolved inclusivity concerns at release. The record also identifies any aspects of diverse character representation that could not be conclusively assessed by the capstone team.

4.2.8 Compliance Requirements

These tests verify the compliance requirements defined in the SRS. They check whether the game and its supporting documentation satisfy the applicable requirements and standards identified for the project.

ST-NFR-33: No Medical Claims in Content and Listings

Type: Non-functional, Static content inspection.

Requirements Covered: NFR32.

Review procedure: Two team members and a clinician will separately review all child-facing text, adult-facing text, and store listings for the final game, including on-screen text, narration, the medical notice, procedure guides, settings and reset screens, and store listing text and screenshots, using a checklist of the game's content. For each item, reviewers will answer the following questions:

1. Does the content claim or imply that the application diagnoses any condition?
2. Does the content claim or imply that the application screens for or detects any condition or problem?
3. Does the content claim or imply that the application treats or cures any condition?
4. Does the content claim or imply that the application monitors or tracks any condition or changes in a child's vision or health?
5. Does the content suggest that the practice can replace a vision test or professional examination, or that a parent can rely on it to find a real problem?

For each question, reviewers will record a Yes/No response. Any "Yes" answer will require a finding containing the content ID, wording, concern, and proposed change. Reviewers will compare findings, record their resolution in GitHub Issues, and reinspect corrected content before release.

Expected result and evidence: Every inventory item has a completed five-question checklist from each of the three reviewers, and there are zero unresolved medical-claim findings for the reviewed release. The saved checklists, content inventory, reviewers' names and roles, build and store listing versions, and GitHub Issue references provide evidence of the review.

ST-NFR-34: Licence Compliance of Third-Party Assets and Libraries

Type: Non-functional, Static inspection of licence records and build contents.

Requirements Covered: NFR33.

Review procedure: The assigned reviewer and a second team member will separately build an inventory of every third-party asset and library in the final release, including code libraries and their dependencies, art, audio, narration, fonts, icons, shaders, and any asset downloaded from a public store. The inventory is taken from the release build and the project's dependency files, not from the licence log, so that items missing from the log can be found. Reviewers then compare the inventory with the licence log. For each item, reviewers will answer the following questions:

1. Does the licence log contain an entry for this item?
2. Does the entry record the licence name or terms, the source, and the version or download date?
3. Does the recorded licence permit redistribution of the item in the application?
4. Is the recorded licence compatible with the project's chosen software licence?
5. Does the entry state any required conditions, such as attribution or notices, and are those conditions met in the build?

For each question, reviewers will record a Yes/No response. Any "No" answer will require a finding containing the item name, source, concern, and proposed change (for example, adding the missing entry, obtaining permission, or replacing the item). Reviewers will compare findings, record their resolution in GitHub Issues, and reinspect corrected items and the licence log before release.

Expected result and evidence: Every inventory item has a completed five-question checklist, the licence log has no gap, and there are zero unresolved licence findings for the reviewed release. The saved checklists, item inventory, licence log, reviewers' names, build version, and GitHub Issue references provide evidence of the review.

ST-NFR-35: Children's App Store & Submission Approval

Type: Non-functional, Static inspection of the release build and store submissions, with device testing.

Requirements Covered: NFR34.

Initial state: The release build, store listings, privacy disclosures, target audience declarations, and policy versions used for submission are frozen.

Input/condition: Review the Apple Kids Category and Google Play Families requirements against the submitted configuration. Cover parental gates, advertising, external links, real-money purchases, permissions, and third-party data practices. Reuse the results of the parental gate tests and the privacy and network traffic review where they apply.

Review procedure: The assigned reviewer and a second team member will build the pre-submission policy checklist from the current text of the Kids Category rules of the App Store Review Guidelines and the Families policy of Google Play. The reviewers will then separately inspect the release build, the store listings, and the store declarations (target audience and age rating answers, privacy labels, and data safety forms) against every checklist item. Inspection includes running the release build on a test device for each store and observing its behaviour, requested permissions, and network traffic. For each item, reviewers will record a Yes/No response, and any "No" answer requires a finding containing the checklist item, policy section, concern, and proposed change. Reviewers will compare findings, record their resolution in GitHub Issues, rebuild the release candidate, and rerun the affected checks. When every item passes, the team has the option to submit the release build to both stores for a children's audience and records each store's review response. Any rejection or request for changes is recorded as a finding and resolved in the same way, and the corrected build is resubmitted.

Expected result and evidence: Every checklist item passes for both stores, signifying that for future submission, both the App Store and Google Play have a high probability of apporving the release build for a children's audience. Evidence is the dated policy checklist with the policy versions, the completed answers from both reviewers, the submitted build identifiers, the records of any correction and the rerun results, the reviewers' names, the GitHub Issue references and in the future, the store review responses and approval records.

4.3 Traceability Between Test Cases and Requirements

Table 5 maps each functional and non-functional SRS requirement to the tests that explicitly cite it in their *Requirements Covered* field, including related checks. Requirement IDs link to the SRS, and test IDs link to the corresponding cases in this plan.

Table 5: Traceability Between Test Cases and Requirements

Requirement	Requirement Focus	Supporting Test Cases
Functional Requirements		
FR1	Medical notice	ST-FR-1, ST-FR-2, ST-FR-48, ST-FR-49
FR2	Stored notice acknowledgement	ST-FR-3, ST-FR-49
FR3	Journey map and level status	ST-FR-4, ST-FR-5
FR4	Unrestricted level access	ST-FR-5, ST-FR-6, ST-FR-7
FR5	Replay completed levels	ST-FR-7
FR6	Procedure content in every level	ST-FR-8
FR7	Age-specific content	ST-FR-9
FR8	Required sensation descriptions	ST-FR-10, ST-FR-14, ST-FR-21, ST-FR-24
FR9	Truthful sensations descriptions	ST-FR-10, ST-FR-14, ST-FR-21
FR10	Differences between game and clinic	ST-FR-11
FR11	Permission to express discomfort	ST-FR-12, ST-FR-14
FR12	Level completion	ST-FR-5, ST-FR-13, ST-FR-14, ST-FR-21
FR13	Narrated instructions and demonstrations	ST-FR-15
FR14	Encouraging hints after errors	ST-FR-16
FR15	Persistent visual cues	ST-FR-17, ST-FR-48
FR16	Instructions without sound	ST-FR-18

Continued on next page

Table 5 – continued from previous page

Requirement	Requirement Focus	Supporting Test Cases
FR17	Correct-answer reveal	ST-FR-19
FR18	Valid questions	ST-FR-20
FR19	No penalties for skips or errors	ST-FR-19, ST-FR-21
FR20	Repeated touches and protected narration	ST-FR-22, ST-FR-23, ST-FR-24, ST-FR-29
FR21	Protected narration and skip timing	ST-FR-22, ST-FR-23, ST-FR-24, ST-FR-29
FR22	Exit to the journey map	ST-FR-25
FR23	Stop level media on exit	ST-FR-26
FR24	Preserve progress on early exit	ST-FR-27
FR25	Start from the beginning after exit	ST-FR-28
FR26	Restart and unexpected-input recovery	ST-FR-25, ST-FR-29, ST-FR-30
FR27	Pause and resume on interruption	ST-FR-31, ST-FR-32
FR28	Save local state	ST-FR-33, ST-FR-34, ST-FR-35, ST-FR-36
FR29	Restore local state	ST-FR-33, ST-FR-34, ST-FR-35, ST-FR-36
FR30	Recover from unusable saved data	ST-FR-37, ST-FR-38, ST-FR-39
FR31	Reward on first completion	ST-FR-21, ST-FR-40, ST-FR-41
FR32	No duplicate rewards	ST-FR-21, ST-FR-34, ST-FR-42
FR33	Personal scoreboard	ST-FR-43, ST-FR-44
FR34	Character selection	ST-FR-45
FR35	Diverse characters and accessories	ST-FR-45
FR36	Accessory shop	ST-FR-46
FR37	Accessory purchases	ST-FR-46
FR38	Refused purchases	ST-FR-46
FR39	Equip and unequip accessories	ST-FR-47
FR40	No real-money purchases	ST-FR-46
FR41	Parental gate for settings	ST-FR-48
FR42	Caregiver procedure guide	ST-FR-48
FR43	Optional nickname	ST-FR-48, ST-FR-49
FR44	Caregiver preferences	ST-FR-48
FR45	Reset journey data	ST-FR-34 (persistence only), ST-FR-49
FR46	Clear all local data	ST-FR-34 (persistence only), ST-FR-49
Non-Functional Requirements		
NFR1	Consistent control positions	ST-NFR-1
NFR2	Child-safe content	ST-NFR-2
NFR3	Touch target size	ST-NFR-3
NFR4	One-touch exit	ST-FR-25, ST-NFR-4
NFR5	Core tasks without training	ST-NFR-5
NFR6	Clinician access to a named level	ST-NFR-6
NFR7	No pressure to keep playing	ST-NFR-7
NFR8	Age-appropriate language	ST-NFR-8
NFR9	WCAG 2.2 Level AA accessibility	ST-NFR-9
NFR10	Cues that do not rely on colour	ST-FR-19, ST-NFR-10
NFR11	Startup time	ST-NFR-11
NFR12	Touch response and reveal timing	ST-FR-19, ST-NFR-12

Continued on next page

Table 5 – continued from previous page

Requirement	Requirement Focus	Supporting Test Cases
NFR13	Animation frame rate	ST-NFR-13
NFR14	Narration and animation timing	ST-NFR-14
NFR15	Safe audio levels	ST-NFR-15
NFR16	Flashing limits	ST-NFR-16
NFR17	Crash-free sessions	ST-NFR-17
NFR18	CPU utilization	ST-NFR-18
NFR19	Offline functionality	ST-NFR-19
NFR20	Installation and first launch	ST-NFR-20
NFR21	Installed application size	ST-NFR-21
NFR22	Clinical approval of content	ST-NFR-22
NFR23	Effort to make likely changes	ST-NFR-23
NFR24	Supported devices and platforms	ST-NFR-24
NFR25	Dependency versions and testing	ST-NFR-25
NFR26	Private local storage	ST-NFR-26
NFR27	Signed releases and official stores	ST-NFR-27
NFR28	No outbound data	ST-NFR-28
NFR29	Minimal stored data	ST-NFR-29
NFR30	No high or critical dependency vulnerabilities	ST-NFR-30
NFR31	Inclusive language and visuals	ST-NFR-31, ST-NFR-32
NFR32	No medical claims	ST-NFR-33
NFR33	Third-party licences	ST-NFR-34
NFR34	Children's app-store policies	ST-NFR-35

5 Unit Test Description

This section will be completed once the design document has been finalized.

5.1 Unit Testing Scope

TBD

5.2 Tests for Functional Requirements

TBD

5.3 Tests for Nonfunctional Requirements

TBD

5.4 Traceability Between Test Cases and Modules

TBD

6 Appendix

6.1 Symbolic Parameters

The first table lists the SRS constants used in these tests. Their values come directly from the SRS. The second table defines the few additional values used to run the tests.

Table 6: SRS Constants

Constant	Value	Description
ADULT_TEXT_MAX_GRADE	8	Maximum reading grade for caregiver text
ANSWER_REVEAL_MAX_S	1	Maximum answer reveal time (s)
AV_SYNC_MAX_MS	100	Maximum audiovisual start offset (ms)
CLINICIAN_FIND_MAX_MIN	1	Maximum time to find content for a patient (min)
CLINICIAN_TRAINING_MAX_MIN	5	Maximum clinician training time (min)
CONTROL_POSITION_TOLERANCE_MM	5	Maximum control position deviation (mm)
CORE_TASK_MAX_MIN	5	Maximum core task duration (min)
CPU_UTILIZATION_MAX_PERCENT	80%	Maximum CPU utilization
CRASH_FREE_RATE_MIN	99%	Minimum crash-free session rate
FLASH_MAX_PER_S	3	Maximum flashes per second
FRAME_RATE_COMPLIANCE_MIN	95%	Minimum time meeting frame-rate target
FRAME_RATE_MIN	30	Minimum frames per second
HEAT_TEST_SESSION_MIN	30	Minimum device heat-test duration (min)
INSTALL_SIZE_MAX_MB	300	Maximum installed application size (MB)
LOW_SPEC_AGE_YEARS	5	Minimum age of low-spec device (years)
MAINT_FRACTION	0.10	Maximum change effort / initial development effort
MIN_ANDROID	10	Minimum supported Android version
MIN_IOS	16	Minimum supported iOS version
RESPONSE_MAX_MS	100	Maximum time to respond to an input (ms)
RESPONSE_RATE_MIN	95%	Minimum quick-response rate
SCREEN_MAX_IN	13	Maximum supported screen diagonal (in)
SCREEN_MIN_IN	4.7	Minimum supported screen diagonal (in)

Continued on next page

Table 6 – continued from previous page

Constant	Value	Description
SENTENCE_MAX_WORDS	10	Maximum words per narrated sentence
SKIP_HOLD_MIN_S	1	Minimum press-and-hold time to skip (s)
STARTUP_MAX_S	5	Maximum median startup time (s)
STARTUP_TRIALS_MIN	10	Minimum cold-start trials per device
TEST_SESSIONS_MIN	100	Minimum test sessions for reliability
TOUCH_TARGET_MIN	15	Minimum touch target size (mm)
UNAIDED_RATE_MIN	80%	Minimum unaided task completion rate

Table 7: Additional System-Test Parameters

Constant	Value	Description
ANSWER_RETRY_OBSERVATION_S	10	Observation after a reveal with no configured timeout; not an allowed timeout (s)
TOUCH_TRIALS_MIN	40	Valid touch samples per device
FREEZE_OBSERVATION_S	10	Time allowed before recording a freeze (s)
COVERAGE_LINE_TARGET	80%	Line coverage target for game-logic assemblies
REPLAY_TRIALS_MIN	3	Replays of a completed level when checking for duplicate rewards
USABILITY_PARTICIPANTS_MIN	5	Participants in the core task usability test
SURVEY_AGREE_RATE_MIN	80%	Agreement below which a survey statement is reviewed

6.2 Reward and Scoreboard Test Data

The Rewards and Scoreboard tests (Section 4.1.7) use the test data below. The data is loaded only in development builds and the Unity editor, and is never included in a release build. Each test level has one skippable exercise and one question with a single correct answer.

Table 8: Test Levels and Their Rewards

Level	Points	Coins	Accessory
L1	10	5	None
L2	20	0	Star badge
L3	40	15	Sun hat

SAVE_MIXED can be reached by normal play. The child completes L1 and L3 for 20 coins and the Sun hat, buys the Blue scarf for 17 coins, and then starts and leaves L2.

Table 9: Saved States for the Rewards and Scoreboard Tests

Saved state	Contents
`SAVE_NEW`	Every level Not Started and 0 coins. No accessories, no selected character, and no nickname.
`SAVE_MIXED`	`L1` and `L3` Completed, and `L2` In Progress. Coin balance of 3. Owns the Sun hat and the Blue scarf, with none equipped. Test character B is selected and the nickname is Sam.

6.3 Design Verification Checklist

This checklist will be used during the structured internal review described in Section 3.3. Reviewers will compare the design document against the SRS and Hazard Analysis. The checklist is subject to change once the design document is finalized.

☐ Are the system's main components identified in the design document, with a clear purpose and responsibility for each one?

☐ Does the design document provide enough detail about each component's inputs, outputs, interfaces, and expected behaviour for another team member to implement it?

☐ Can the functional and non-functional requirements in the SRS be traced to the components or design decisions that support them?

☐ Does the design cover the main gameplay flows, including selecting a level, completing it, saving progress, and exiting early?

☐ Does the design account for the safety-related behaviour in the Hazard Analysis, such as avoiding punitive feedback, preventing accidental input, and stopping audio and animations when a level is exited?

☐ Is the design divided into components that can be tested and updated without unnecessary changes to unrelated parts of the game?

☐ Are external packages and assets identified so their licensing and any relevant security or content concerns can be checked?

☐ Are diagrams, interface descriptions, and assumptions clear and consistent enough for the team to implement and test the design?

6.4 Usability Survey Questions

Participants will complete this survey straight after ST-NFR-5 or ST-NFR-6, before a team member discusses the session with them. The survey records how the game seemed to them. It adds evidence for NFR1–NFR10 alongside the measured results, and does not replace any fit criterion. Participants will answer on behalf of a child, so the results can identify potential problems but cannot confirm how children would respond (Section 2.2).

Before the survey, a team member will open the caregiver area so that each participant can read one text. The form will not record names, email addresses, or other personal information. Before the first session, one classmate who is not a test participant will pilot the survey, and any statement they find unclear will be reworded.

Rating Statements

Participants will rate each statement in Table 10 from 1 (strongly disagree) to 5 (strongly agree), or choose *Not seen* if they did not see that part of the game. Participants in ST-NFR-5 will answer as the persona they acted and will skip the clinical staff statements. Participants in ST-NFR-6 will answer for the parts of the game they saw and all of the clinical staff statements.

Table 10: Usability Survey Rating Statements

ID	Statement	Requirement
Look and Feel		
SQ1	Controls with the same purpose, such as the exit and hint buttons, stayed in the same place on every screen.	NFR1
SQ2	I knew where to look for a control without searching the screen.	NFR1
SQ3	The images and animations would not frighten a child aged 3 to 7.	NFR2
SQ4	The narration and sounds would not frighten a child aged 3 to 7.	NFR2
SQ5	The game showed the procedure in a calm and reassuring way.	NFR2
Usability and Humanity		
SQ6	The buttons and other touch areas were large enough for a young child to touch accurately.	NFR3
SQ7	It was clear how to leave a level at any point.	NFR4
SQ8	I could start the level I was asked to find without instructions.	NFR5
SQ9	In each exercise, I knew what to do next without help.	NFR5
SQ10	The game did not pressure me to keep playing or to come back.	NFR7
SQ11	The narrated sentences were short enough for a child aged 3 to 7 to follow.	NFR8
SQ12	The narration used words that a child aged 3 to 7 would know.	NFR8
SQ13	The text in the caregiver area was easy to understand.	NFR8
SQ14	The text and icons were easy to see on this screen.	NFR9
SQ15	I could follow each exercise from the visual cues without relying on the narration.	NFR9
SQ16	I could tell the level statuses on the journey map apart without relying on colour.	NFR10
SQ17	I could tell which answer was correct after it was revealed without relying on colour.	NFR10
Clinical staff only		
SQ18	Finding a named level would be fast enough during an appointment.	NFR6
SQ19	The training I received was enough for me to show the game to a patient.	NFR6

Open-Ended Questions

OQ1: Which screen or control was the hardest to find or use, and why?

OQ2: Which image, sound, word, or sentence might upset or confuse a child aged 3 to 7, and why?

OQ3: Did anything make you feel rushed or pressured to keep playing? If so, what was it?

OQ4: What single change would most help a child use the game without help?

OQ5: (Clinical staff only) What would stop you from using the game during an appointment?

Use of the Results

The VnV Report will present the classmate and clinical staff results separately. For each statement, it will give the median rating and the number and percentage of participants who rated it 4 or 5, excluding *Not seen*. A diverging stacked bar chart will show the responses to each statement, grouped by requirement. With few participants, these values describe the group and will not be treated as statistically significant.

The survey does not pass or fail a requirement. If fewer than `SURVEY_AGREE_RATE_MIN`% of participants rate a

statement 4 or 5, the team will open a GitHub Issue and review the statement against the related test results and session notes. Open-ended answers will be grouped by screen, and any problem reported by more than one participant will also be recorded in GitHub Issues.

6.5 SRS Checklist Questions

The SRS Checklist in Table 11 identifies key items to check and verify the SRS against the project intentions and the course rubric. The checklist items are categorized on verification goals:

- Clarity: The reader can understand the content without ambiguity.
- Correctnes: The content is accurate for the domain and the projects intent.
- Completeness: Nothing required or expected is missing.
- Consistency: Content does not conflict with itself or restate itself inconsistently.
- Traceability: Requirements link to their sources and dependents.
- Quality: The document is professionally written, organized, and suitable for its audience.
- Testability: Each requirement can be checked by a measurable test.

Each row records the following checklist item:

- Item: The checklist item description.
- Verification Goal: For redundancy, the specific verification goal the checklist item traces to.

Table 11: SRS Checklist Items and Their Verification Goal for Justification

	Checklist Item	**Verification Goal**
☐	The purpose and scope (Sections 1.1, 1.2) make clear what EyeCanDo does and does not do.	Clarity
☐	Every term and acronym is defined before first use, and Section 1.4 expands all acronyms with links to where they are defined.	Clarity
☐	Wording is clear and precise, so two readers would interpret each requirement the same way.	Clarity
☐	The document is clearly structured, with correct grammar, spelling, and hyperlinks, and a new team member could use it to get up to speed.	Clarity
☐	The stakeholders and user characteristics are correct and complete, and the personas and use cases are realistic for children aged 3 to 7, caregivers, and clinicians.	Correctness
☐	Verification Task 1: Pick two use cases (suggest one of UC5, UC6, UC7). The steps, equipment, and sensations match real clinical practice.	Correctness
☐	Every non-obvious requirement has a rationale, including the specific values used (e.g., thresholds and time limits).	Correctness
☐	All required elements are present and of high quality: scope, symbolic constants, stakeholders, user characteristics, context diagram, stakeholder requirements, personas, use cases, FRs, data requirements, NFRs, rationale, likely and unlikely changes, generative AI disclosure, and reflection.	Completeness
☐	The data dictionary (Section 7.2) covers all data the FRs and use cases mention.	Completeness
☐	Likely and unlikely changes are documented, and the unlikely ones are justified.	Completeness

Continued on next page

Table 11 – continued from previous page

	Checklist Item	Verification Goal
☐	An outsider could design, build, and test from this document, and the inputs and outputs are clear. Open decisions (e.g., "level format not yet decided") are justified and bounded.	Completeness
☐	Terminology is used consistently (e.g., level vs. game, journey points vs. coins, home control).	Consistency
☐	No two requirements conflict. (Where requirements only look similar, such as FR23 vs. FR25 on restart vs. resume, check that the distinction is clear.)	Consistency
☐	The formal model (Section 6.1) matches the FRs and use cases that rely on it.	Consistency
☐	Symbolic constants are defined in one place, used consistently, and each value is justified (Appendix C).	Consistency
☐	Every requirement has a unique identifier, and every cross-reference points to the correct item.	Traceability
☐	Traceability is complete and bidirectional: every STR is satisfied by at least one UC, FR, NFR, or constraint; every UC and FR traces to an STR. If A cites B, B cites A and the matrices in Section 9 match the in-text "Traces" and "Refined by" lines.	Traceability
☐	Requirements state what, not how. Any externally imposed design decision is documented as a constraint with rationale.	Quality
☐	The SRS shows evidence of consulting real users or clinicians, and shows what makes EyeCanDo different from existing preparation options.	Quality
☐	Each requirement has a measurable fit criterion, with no vague wording like "fast" or "easy."	Testability
☐	Verification Task 2: Pick two or three requirements (e.g., FR8, FR19, an NFR). The reviewer can describe how they would test each one.	Testability

6.6 Research Proxy to Stakeholder Requirement Validation

Table 12 identifies the research source used demonstrating validation that a stakeholder requirement defined in the SRS can be achieved by EyeCanDo. Each row includes:

- Stakeholder Requirement: The stakeholder requirement code corresponding to its requirement in the SRS.
- Evidence: The research resource citation.
- Justification: A brief description justifying how the research validates the stakeholder requirement.

Table 12: Research Evidence Mapping to Stakeholder Requirement for Validation

Stakeholder Requirement	Evidence	Justification
STR1	Interactive video games to reduce paediatric procedural pain and anxiety: a systematic review and meta-analysis, Mona F Sajeev [5]	Sajeev et al. [5] conducted a systematic review and meta-analysis of 36 studies (3,406 children aged 0 to 18) comparing interactive video games with standard care during painful medical procedures. Pooled results showed a reduction in children's procedural pain (standardized mean difference −0.43, 95% CI −0.67 to −0.20), child anxiety (−0.61, 95% CI −0.88 to −0.34), and caregiver anxiety (−0.31, 95% CI −0.58 to −0.04). Each confidence interval lies entirely below zero, so the reductions are unlikely to be due to chance. The pain and caregiver effects are small to medium, and the child anxiety effect is medium. The review found no detectable difference between preparatory and distraction games, or between virtual reality and non-virtual reality games. This supports the use of simple, widely available games to reduce distress in children.
STR2		
STR3		
STR4		
STR5		
STR6		
STR7		
STR8		
STR9		
STR10		
STR11		
STR12		
STR13		
STR14		
STR15		
STR16		
STR17		
STR18		

6.7 Software Validation Checklist for Supervisor Validation

Table 13 presents the questions to ask the supervisor to validate EyeCanDo in the context of the stakeholder requirements and use cases developed against the clinical needs. Each row includes:

- Question: The question to validate particular stakeholder requirements and use cases.
- Stakeholder Requirement: The stakeholder requirement being validated.

- Connected Use Cases: The use cases being validated by the question.

Table 13: Supervisor Validation Questions for the Rev 0 Demo - Version 1

Question	**Stakeholder Requirement**	**Connected Use Cases**
Does each procedure match what children experience at your clinic, including the equipment, steps, and sensations described?	STR1, STR5, STR6	UC5, UC6, UC7
Is anything shown, said, or implied likely to frighten or mislead a child aged 3 to 7?	STR2, STR15	UC5, UC6, UC7
Is the language and narration appropriate for the 3 to 4 and 5 to 7 age groups?	STR3, STR18	UC5, UC6, UC7
Could you introduce this to a patient within the time available in an appointment?	STR7, NFR6	UC3, UC4
Would caregivers understand what the app is for and what it is not, from the notice and the procedure guide?	STR14	UC1, UC4
Does the scoreboard show what you and the evaluation study need to know about which activities a child has completed?	STR9	UC9, UC10
Are the rewards, coins, and accessories appropriate, without encouraging compulsive use or pressure?	STR15, STR16	UC9, UC11, UC12
Is the app usable in a clinic waiting room, offline and with the sound muted?	STR4, STR10	UC15

7 Appendix — Generative AI Use Disclosure

7.1 AI Tools Used

- ChatGPT (GPT-6.1 Sol, GPT-6)
- Claude (Opus 5.5)

7.2 How and Where Used

- In Section 2 (General Information), GPT-6 was used to assist with organizing, wording, and formatting the Summary, Objectives, Custom Documents, and Relevant Documentation subsections. The content was reviewed to ensure consistency with the SRS, project scope, and decisions made by the team.
- In Section 3.1 (Verification and Validation Team), GPT-6 was used to improve the organization, wording, and formatting of the team responsibilities and testing assignments. It also assisted with the LaTeX formatting of the team roles table. The assigned roles and responsibilities were based on the team's agreed division of work.
- In Section 3.2, Opus 5.5 was used to assist with drafting the SRS verification. Opus also assisted in the formatting and organization of the SRS verification section for clarity. The content was reviewed and finalized by hand to ensure the accuracy and completeness of the SRS verification process. In Section 6.5, Opus 5.5 was used to assist with drafting the SRS checklist questions based on the SRS and the course rubric. The content was reviewed, edited,and finalized by hand to ensure the accuracy and completeness of the SRS checklist questions.
- In Section 3.3, the team developed the design verification approach. GPT-6.1 Sol was used to improve the wording, grammar, and flow while preserving the team's original ideas and decisions.
- In Section 3.4 (Verification and Validation Plan Verification), GPT-6 was used to write the initial draft of the checklist, improve its wording, and provide feedback that the team member then implemented.
- In Section 3.5 (Implementation Verification), GPT-6 was used to write the initial draft, improve the wording, and ensure consistency with related source documents, including the Development Plan. The team member then implemented its feedback.
- In Section 3.7, Opus 5.5 was used to assist with the formatting and organization of the software validation section for clarity. The content was reviewed and finalized by hand to ensure the accuracy and completeness of the software validation process with respect to the project. In Section 6.6, Opus 5.5 was used to format Table 12 and assist with drafting of the justification for the resource by Sajeev et al. [5]. The content was reviewed, edited, and finalized to ensure the correctness of the research paper justification. In Section 6.7, Opus 5.5 was used to draft the validation questions and their traces to SRS stakeholder requirements and use cases. Opus 5.5 was also used to format Table 13. All of this work was reviewed, edited, and finalized by hand to ensure the accuracy and completeness of the validation questions with respect to the project.
- In Section 4.1.1, 4.1.2, 4.1.3 (Tests for Functional Requirements), GPT-6 was used to assist with drafting, organizing, and formatting the functional system tests for Setup, Journey Map, and Level Content (FR1–FR12). This included structuring the test conditions, initial states, expected results, test case derivations, and execution procedures. The tests were reviewed against the SRS to ensure appropriate requirement coverage, traceability, and feasibility within the capstone scope.
- In Sections 4.1.4–4.1.6 (Instructions and Feedback, Exiting and Interruptions, and Saving and Restoring Progress), GPT-6 was used to write the initial drafts, improve the wording, and ensure consistency across all source documents for those test cases. The team member then implemented its feedback.
- In Section 4.1.8 and Section 4.1.9, Opus 5.5 was used to assist with drafting and formatting the of the tests. The content was reviewed and edited to ensure the tests accurately reflected the SRS requirements and were feasible within the capstone scope.

- In Section 4.2.7, GPT-6 was used to assist with drafting and formatting the Cultural and Political Requirements tests covering NFR31 (Inclusivity). This included organizing the static review procedures, inspection checklists, expected results, and evidence to be collected. The tests were reviewed against the SRS and Hazard Analysis.

- In Sections 4.2.3–4.2.6, GPT-6.1 Sol was used to draft the non-functional system tests. The team then reviewed and revised the test procedures, expected results, and scope remarks to align with the SRS and the resources available for the capstone.

- In Section 4.2.8, Opus 5.5 was used to assist with drafting and formatting the Compliance tests, including the checklists within the review procedure. The content was reviewed and edited to ensure the tests accurately reflected the SRS requirements and the fit criterion specified within the non-functional requirements in the SRS.

- In Section 4.3 (Traceability Between Test Cases and Requirements), GPT-6 was used to create the table's LaTeX formatting, ensure all test cases and their requirement relationships were captured, and fix broken requirement links as needed.

- In Section 6.1, GPT-6.1 Sol was used to help organize the symbolic parameter tables and link constants to the tests. The team verified the values against the SRS.

- In Section 6.3, GPT-6.1 Sol was used to draft the design verification checklist, which the team then reviewed and revised.

- In Section 3.6, Claude (Opus 5.5) was used to draft the tool summary and the coverage plan from the Development Plan and the team's issue. The team checked the tool choices against the Development Plan and confirmed which tools are still under evaluation.

- In Section 4.1.7 (Rewards and Scoreboard), Claude (Opus 5.5) was used to draft the functional system tests for FR31–FR33 and the test data in Appendix 6.2. It also added the `REPLAY_TRIALS_MIN` parameter. The team member directed the revisions, such as one test per requirement, automated checks of the saved game state, and simpler test data. The tests and expected values were checked against the SRS.

- In Sections 4.2.1 and 4.2.2 (Look and Feel, and Usability and Humanity), Claude (Opus 5.5) was used to draft the non-functional system tests for NFR1–NFR10, following the format of the other non-functional tests. It also added the `USABILITY_PARTICIPANTS_MIN` parameter. The team member reviewed the procedures and remarks against the SRS fit criteria and the capstone limits on testing with children and clinical staff.

- In Appendix 6.4 (Usability Survey Questions), Claude (Opus 5.5) was used to draft the survey statements, the open-ended questions, and the plan for using the results, based on the course checklist and NFR1–NFR10. It also added the `SURVEY_AGREE_RATE_MIN` parameter. The team member reviewed the survey against the SRS.

- In Appendix 8 (Reflection), GPT-6 was used to improve the wording, clarity, and conciseness of Jackson's reflection.

- GPT-6.1 Sol & GPT-6 were also used to assist with automatic test numbering, cross document references, and general LaTeX formatting.

7.3 Verification of AI-Generated Content

The team reviewed the AI-assisted content against the SRS, Hazard Analysis, design documents, and the scope of the project. The proposed verification and validation activities were checked to ensure that the selected methods were appropriate, the roles and review procedures were clear, and the planned evidence would be sufficient to assess the requirements. The system and unit test plans were checked for requirement coverage, traceability, clearly defined inputs and expected results, and feasibility within the time and equipment available to the team. Where a test could not fully meet the SRS fit criterion within the capstone scope, the limitation was stated explicitly.

The team also checked AI-suggested thresholds, symbolic parameters, and requirement references against the source

documents. Checklists and review procedures were examined for clarity and completeness. Finally, the LaTeX document was reviewed for correct numbering, cross-references, tables, citations, and formatting. The team remains responsible for the accuracy, completeness, consistency, and appropriateness of the final VnV Plan.

8 Appendix — Reflection

Questions 1 and 2 were answered individually by each team member. For Questions 3 and 4, the team brainstormed the knowledge and skills needed to complete the VnV activities and possible ways to learn them. Each member then selected one skill and reflected on the approaches they would use to develop it. The individual responses are provided below.

Mohammad Bilal

1. **What went well while writing this deliverable?**

   One thing that went well was coming up with the overall approach for the system tests. When writing the SRS, we tried to make each requirement distinct and avoid unnecessary overlap. This was especially helpful for the NFRs, which I worked on in both the SRS and this document. Since the requirements were already fairly clear and separate, I could plan one test for each NFR in my assigned sections. That gave me a good starting point, and I could then focus on working out the test conditions, expected results, and how each test would actually be carried out.

   Another thing that went well was dividing the work early, as we have done for previous deliverables. Unlike the SRS, most sections of the VnV Plan did not depend heavily on one another, so team members could make progress independently and at their own pace. Assigning responsibilities early also made it easier to stay organized and manage the limited time we had for the deliverable.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**

   One challenge was figuring out how we could validate the game with its intended users. Direct testing with paediatric patients would require Research Ethics Board (REB) approval, and our supervisor explained that obtaining this approval would likely be infeasible within the capstone timeline. We discussed the issue with our TA and course instructor and adjusted our validation plans accordingly. Instead of relying on direct testing with children, we will need to use feedback from ophthalmologists and other people who work with the intended users, along with simulated activities where appropriate. This is not quite the same as testing with the children themselves, so it is an important limitation of our plan.

   Another challenge was the number of tests we needed to plan. The SRS has 43 functional requirements and 29 non-functional requirements, and writing a meaningful test for each takes time. It was not enough to simply restate the requirement; we had to decide what conditions to test, how to measure the result, and whether the test was realistic with the tools and devices available to us. AI helped generate some starting ideas, but many of its suggestions were too generic or involved testing resources we would not have. I had to revise those suggestions and work through the details manually to make the tests specific and achievable for our project.

3. **What knowledge and/or skill will you need to acquire to successfully complete the verification and validation of your project?**

   The team identified several skills that we will need for verification and validation, but the one I want to focus on is learning how to use the Unity Profiler. Many of the non-functional tests I worked on require actual measurements, particularly for frame rate, response time, CPU usage, and memory use. I need to learn which measurements to collect, how to record them consistently during gameplay, and how to interpret the results. For example, I should be able to tell whether a drop in frame rate is a brief issue while loading a level or a problem that continues during gameplay. I also want to understand the difference between profiling in the Unity Editor and profiling a build on a device, since the Editor itself can affect performance. Learning this will help me carry out the NFR tests properly and provide evidence for whether the game meets the SRS targets, especially with the limited physical devices available to our team.

4. **What approaches will you pursue to acquire this knowledge or master this skill, and why?**

   My first approach will be to work through the official Unity Profiler documentation and its examples, focusing on the CPU Usage, Memory, and frame timing tools. The documentation should help me understand how to

capture a profiling session and what the different measurements mean. I will then practice using the Profiler on our game as development progresses, such as by recording a playthrough of a level and looking at what happens when animations or scene transitions occur. This will give me experience collecting and comparing results before we run the final NFR tests.

My second approach will be to use AI when I have a specific question or run into a problem with the Profiler. For example, it could help me understand an unfamiliar measurement, troubleshoot a setup issue, or suggest what to check when performance results seem inconsistent. This would be a quick way to get unstuck while learning the tool. However, I would still check any suggestions against Unity's documentation and test them myself rather than assuming the AI explanation is correct. Using both approaches should help me learn the Profiler more efficiently while making sure the measurements we use in the VnV Plan are reliable.

Ahsan Muzammil

1. **What went well while writing this deliverable?**

   One thing that went well was how quickly our team was able to divide the work and establish clear responsibilities for the VnV Plan. Early in the process, we identified the sections and requirement categories that needed to be completed and assigned them to team members through GitHub issues. This made it easier for everyone to understand what they were responsible for and what was expected of them for the deliverable.

   Using GitHub issues also helped us track everyone's progress and monitor how much work each person had been assigned. This was particularly useful for ensuring that responsibilities were distributed fairly and that no individual was taking on a disproportionate amount of work. It also gave us a way to identify incomplete tasks and follow up on them before the deadline.

   Personally, I was able to complete my assigned sections and functional system tests in a timely manner with minimal conflicts. Since the responsibilities were clearly defined, I could work independently without frequently needing clarification from other team members. Overall, our approach to task allocation and progress tracking made the writing process more organized and efficient.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**

   One of the main challenges we experienced was a disconnect between our understanding of the capstone course expectations and our supervisor's expectations regarding communication, stakeholder involvement, and the scope of the project. This created uncertainty about how we could proceed with certain development and validation activities, particularly those involving our intended users.

   A significant concern was that directly involving children and caregivers in usability testing or interviews would require Research Ethics Board (REB) approval. Our supervisor explained that pursuing this approval would not be feasible within the capstone timeline. As a result, we could no longer proceed with the Client Interview Report that we had originally considered for our Fall Custom Document. We also became uncertain about which Custom Documents would be appropriate for the Winter term without access to our primary stakeholders.

   To address these issues, we discussed our concerns with our TA, Rashad, who recommended that we communicate our situation and questions to the course staff for further guidance. We also began reconsidering our planned Custom Documents, identifying a Cognitive Walkthrough Report or Wireframe Report as possible alternatives for the Fall semester. For the VnV Plan, we adjusted our proposed validation activities to focus on internal reviews, peer evaluations using child and caregiver personas, and feedback from our supervisor where feasible.

   Although these changes allowed us to continue planning the deliverable, some uncertainty remains regarding the final selection of our Custom Documents and the validation activities we can realistically perform. This experience highlighted the importance of establishing clear expectations with supervisors early in the project and communicating potential constraints before they affect planned deliverables.

3. **What knowledge and/or skill will you need to acquire to successfully complete the verification**

and validation of your project?

One skill I will need to develop is the ability to implement and execute reproducible functional tests using Unity and its testing tools. For the VnV Plan, I was responsible for planning the functional system tests covering Setup, Journey Map, and Level Content (FR1–FR11), as well as the inclusivity requirement (NFR31). While I was able to define the test conditions, expected results, and execution procedures, carrying out these tests will require a better understanding of how to verify application behaviour consistently.

In particular, I want to learn how to use the Unity Test Framework and Play Mode tests to verify interactions such as starting a preparation level, updating its completion status, replaying a completed level, and preserving progress between application launches. I also need to understand how to create repeatable initial conditions, such as resetting saved data before a test, so that results are not affected by previous gameplay sessions.

Some of my assigned requirements involve educational content and will still require manual inspection. Therefore, I also want to improve my ability to combine automated functional checks with structured manual testing and record evidence such as screenshots, test results, and observed behaviour. Developing these skills will help ensure that our tests provide reliable evidence that EyeCanDo meets its functional requirements.

4. **What approaches will you pursue to acquire this knowledge or master this skill, and why?**

   My first approach will be to study the official Unity Test Framework documentation and work through examples of Edit Mode and Play Mode testing. I will focus on understanding how tests are structured, how assertions are used to compare actual and expected results, and how Unity handles scene loading and application state during testing. This will provide a foundation for determining which of my assigned functional requirements can be verified automatically and which require manual interaction on a physical device.

   My second approach will be to practice writing and executing tests directly within our EyeCanDo project as development progresses. I plan to begin with relatively simple behaviours, such as verifying that a preparation level can be selected and started regardless of the completion status of other levels. I can then work toward testing more complex behaviours, including saving progress, preserving completed statuses during replay, and resetting application data to reproduce first-launch conditions. Running these tests against the actual application will help me identify limitations in my understanding and improve the reliability of the test procedures.

   I will also review my testing approach and results with other team members to confirm that the expected behaviour matches the SRS and that the tests are sufficiently clear for someone else to reproduce. Combining documentation-based learning with hands-on practice and peer feedback will help me develop the skills needed to execute my assigned VnV tests effectively and document trustworthy results.

Jake Finlay

1. **What went well while writing this deliverable?**

   Once again, what went best was splitting up the work early using GitHub issues. I have said this in previous reflections, but it makes a real difference. Everyone knew what was going on and who was doing what, so the project moved much more smoothly.

   Another thing that went well was the improved test case template that Ahsan committed. It gave me a clear format to follow, which made it much easier to write my tests and contribute to both the functional and non-functional sections.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**

   My main pain point was verifying everything against the SRS. Even with AI to help us, every test still had to be checked to confirm that it was testing the correct requirement. To make this easier, I linked the tests directly to the SRS using the `#` syntax in our links. Clicking a link opens the SRS at the exact position of the requirement. This made it much easier to verify what each test covered and which requirements were being verified.

3. **What knowledge and/or skill will you need to acquire to successfully complete the verification and validation of your project?**

   The team identified several skills that we will need, but the one I want to focus on is accessibility auditing. One of the tests I planned is the WCAG 2.2 Level AA audit (ST-NFR-9). It requires us to decide which success criteria apply to a native mobile game and to justify each criterion we leave out. This is not straightforward because WCAG was written mainly with web content in mind, and I have not carried out a formal accessibility audit before.

   I also need to learn how to check each criterion reliably on our test devices. This includes measuring the contrast of text and controls and using VoiceOver and TalkBack to confirm that labels are read correctly. I also need to understand how these screen readers interact with a Unity interface, since Unity does not draw its interface with standard native controls. Learning this will help me produce audit results that a second team member can reproduce.

4. **What approaches will you pursue to acquire this knowledge or master this skill, and why?**

   My first approach will be to study the W3C documentation, including the Understanding WCAG 2.2 guide and the guidance on applying WCAG to non-web software. These are the authoritative sources for what each criterion means. They should help me build a list of applicable criteria that I can justify, rather than relying on my own interpretation.

   My second approach will be to practise auditing early builds of EyeCanDo as development progresses. I plan to begin with a single screen, using a contrast checker and then a screen reader on a physical device. I can then compare my findings with those of a second team member and discuss any differences. Hands-on practice will show me where the guidelines are hard to apply to a game, and it will expose problems while they are still cheap to fix. Combining these approaches should help me carry out the final audit accurately and give us evidence we can trust.

Ethan McMehen

1. **What went well while writing this deliverable?**

   I found that the work was well organized and divided among the team members. Each member had a clear understanding of their responsibilities, which allowed for efficient progress on the deliverable. Additionally, the use of AI tools to assist with drafting and formatting helped streamline the writing process and ensured consistency across sections. Specifically, I found it especially useful in Section 3.2 SRS Verification and Section 3.7 Software Validation because it helped me to clarify my plans and it assisted in generating the drafts for the required checklists and tables. I then was able to have a starting point for those checklists, so I cross-referenced the SRS with my plan to ensure I included questions or refined questions for the checklists. I also found writing the tests was pretty straightforward because the SRS was well defined and the requirements were clear. I was able to use the SRS as a reference to ensure that my tests were accurate to the requirements they are verifying.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**

   I had two major pain points. The first pain point was that I needed to actually make checklists for the SRS Verification and Software Validation sections. To do this completely by hand at first would take a lot of time. Additionally, achieving good accuracy would be difficult because I would have to completely cross-reference everything from scratch without a reference or starting point for traceability.

   The second paint point involved Section 3.7 Software Validation, and specifically the first proxy I have which is the research resources. Since the template mentioned "If there is any external data that can be used for validation, you should point to it here", I would have to find research resources that would validate the stakeholder requirements. This was a pain point because I had to find research resources that were relevant to the stakeholder requirements and then justify how they validated the stakeholder requirements. But this does not seem correct or feasible for a verificatio and validation plan because at this stage, we are just making a plan and not actually carrying out the validation. Therefore, I was confused as to whether I should fill our

Table 12 with research resources right now or if I should leave the table as a template.

I was able to resolve my first pain point by using Claude Opus 5.5 to give me starting lists for the checklists. I did this by passing in the SRS document and VnV Plan Course rubric and then describing my goal. Then after a first draft, I had a goal to achieve and I used it to guide actually generating more specific and meaningful checklists. To solve my second pain point, I communicated with my team about my dilemma and they helped me to decide to send a message in Teams chat and email our TA. Our TA responded to me the morning of the deliverable deadline which was perfect timing! He advised to include at least one article to demonstrate in the template how the validation plan would actually be executed. With this in mind, I found a meta-analysis article and used it to fill out Table 12 and justify how it validated the stakeholder requirements STR2.

3. **What knowledge and/or skill will you need to acquire to successfully complete the verification and validation of your project?**

   The knowledge I would like to acquire to successfully complete the verification and validation of our project is data elicitation for research and App Store requirements. I previously worked in a business of mine where I built and app and worked through App Store approval. While I have some experience with App Store requirements, I would like to learn more about the specific requirements for our project and how to elicit data from research resources to validate our software for deployment. This knowledge will help me ensure that our project meets the necessary standards and requirements for successful deployment and use. Additionally, this knowledge will require elicitation of data from research resources, stakeholder meetings, cognitive walkthroughs, and other methods to ensure that our software meets the needs of our intended users. This knowledge will be crucial for ensuring that our project is validated in the eyes of our stakeholders.

4. **What approaches will you pursue to acquire this knowledge or master this skill, and why?**

   As Team Liaison I can optimize my approach to acquiring this knowledge by leveraging my position to communicate with our stakeholders and gather feedback on our project. I will be highly integrated into the verification and validation meetings with stakeholders because I worked on the SRS Verificaton and Software Validation plans, so I currently have the best understanding on how to execute these elements. Therefore, direct communication with stakeholders will be a primary approach to acquire this knowledge. Once I have the knowledge, I can work further by researching data from papers and publications to validate our software futher. Altogether, this data will serve as a strong foundation for the App Store requirements checklists to ensure an application can be approved.

Jackson Beach

1. **What went well while writing this deliverable?**

   What went well was that our source documents gave me a clear starting point for my sections. The SRS outlined the expected behaviour, the Hazard Analysis identified failure cases to consider, and the Problem Statement kept the project's purpose and scope clear. This made it relatively straightforward to extract the information needed for my test cases. The Development Plan was also helpful for implementation verification, since we had already agreed on our review process, coding standards, and testing tools. Having these decisions written down made it easier to describe how we would verify the implementation.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**

   The main pain point I found was keeping track of the information across all of these documents. There was a lot to work through, and one test could depend on several requirements, safety considerations, and development decisions. It became tricky to keep those connections clear and make sure the expected results were consistent with the sources. I addressed this by cross-checking the tests against the related sections and reviewing their links in the traceability table. I also used AI to help identify gaps and inconsistencies, then checked its suggestions against the source documents before making changes.

3. **What knowledge and/or skill will you need to acquire to successfully complete the verification and validation of your project?**

One skill I need to develop is controlled fault injection, which means deliberately causing a failure to check how the application handles it. This is relevant to my tests for saving and restoring progress, where I need to check what happens if a write fails, saving is interrupted, or saved data is invalid. I need to learn how to trigger these conditions at a specific point and confirm that the intended failure actually occurred. This will help me determine whether the game preserves or resets progress as required, and whether it remains usable afterwards.

4. **What approaches will you pursue to acquire this knowledge or master this skill, and why?**

   My first approach will be to build a small save-and-load example using disposable data. I will add controls that can delay a write or force it to fail, then compare the saved values before and after each run. I will also prepare copies with one invalid field at a time and keep a valid copy for comparison. Starting with a small example will make it easier to understand each failure before applying the approach to the game.

   I will also practise using the debugger to pause at different points in the saving process. Before each run, I will write down what I expect to be restored if the process ends at that point, then compare that prediction with the data loaded after restarting. This will help me understand when a change has actually been saved and where recovery can go wrong. Once I can reproduce these cases reliably, I will apply the same approach to our planned tests, keeping the fault controls in development builds.

References

[1] Team 2, Hindsight. Software Requirements Specification for EyeCanDo: An Ophthalmology Preparation Game for Children. https://team-hindsight.github.io/ophthamology-game/SRS/SRS.pdf, 2026. McMaster University Software Engineering Capstone Project.

[2] Team 2, Hindsight. Problem Statement and Goals: EyeCanDo. https://team-hindsight.github.io/ophthamology-game/ProblemStatementAndGoals/ProblemStatement.pdf, 2026. McMaster University Software Engineering Capstone Project.

[3] Team 2, Hindsight. Development Plan: EyeCanDo. https://team-hindsight.github.io/ophthamology-game/DevelopmentPlan/DevelopmentPlan.pdf, 2026. McMaster University Software Engineering Capstone Project.

[4] Team 2, Hindsight. Hazard Analysis: EyeCanDo. https://team-hindsight.github.io/ophthamology-game/HazardAnalysis/HazardAnalysis.pdf, 2026. McMaster University Software Engineering Capstone Project.

[5] M. F. Sajeev, L. Kelada, A. B. Y. Nur, C. E. Wakefield, M. A. Wewege, J. Karpelowsky, B. Akimana, A.-S. Darlington, and C. Signorelli. Interactive video games to reduce paediatric procedural pain and anxiety: A systematic review and meta-analysis. *British Journal of Anaesthesia*, 127(4):608–619, October 2021.